RESEARCH-ARTICLE

BWAFSQLi: Bypassing Web Application Firewall with Adversarial SQL Injections

BING ZHANG, Yanshan University, Qinhuangdao, Hebei, China

CHAO LIU, Yanshan University, Qinhuangdao, Hebei, China

RONG REN, Yanshan University, Qinhuangdao, Hebei, China

QIAN WANG, Yanshan University, Qinhuangdao, Hebei, China

JIADONG REN, Yanshan University, Qinhuangdao, Hebei, China

Published: 19 January 2026
Accepted: 05 January 2026
Revised: 08 November 2025
Received: 18 July 2025

BWAFSQLi: Bypassing Web Application Firewall with Adversarial SQL Injections

BING ZHANG*, CHAO LIU, RONG REN, QIAN WANG, and JIADONG REN, School of information science and engineering, the Key Laboratory for Software Engineering of Hebei Province, Yanshan University, China

SQL injection-based adversarial attacks can directly evaluate WAFs by observing block/allow actions, yet existing methods have four key issues: low quality and diversity of payloads, inadequate mutation strategies, semantic inequivalence of mutated payloads, and inefficient search processes for generating such payloads. We hypothesize that a method simultaneously improving these aspects would yield more effective attacks. Thus, we propose BWAFSQLi, a general and extensible framework for adversarial SQLi-based WAF bypass. It first designs a convergence-factor-guided context-free grammar to generate high-quality, diverse payloads (covering 18 attack scenarios, targeting 58 rules). For detected payload tokens, BWAFSQLi applies 26 rules with 15 mutation strategies—including two novel techniques (Quotation Mark Encoding and Comment Extension)—to ensure semantic-equivalent mutations. A mutation strategy selection mechanism, integrating a decay factor and historical data table, enables adaptive multi-position mutations for efficient exploration while reducing requests. Mutated payloads are finally evaluated via HTTP requests against target WAFs. Experiments with one self-built dataset (SQLiCFG) and two public datasets (HPD, SIK) on 11 WAFs (3 gray-box, 8 black-box) show BWAFSQLi increases WAF's false negative rates (FNR) by up to 93.39% (gray-box) and 58.49% (black-box) with minimal-requests, surpassing three SOTA methods. Applying seven proposed preprocessing defenses fully suppresses FNR increases, highlighting practical significance.

CCS Concepts: • **Security and privacy → Software and application security**.

Additional Key Words and Phrases: SQL Injection, Web Application Firewall, Context-free syntax, Mutation payloads, Decay Greedy, Weighted Mutation

1 Introduction

In today's complex threat landscape, WAFs must remain vigilant to accurately identify and block diverse attacks [15, 24, 28]. However, as attack techniques evolve—particularly SQL injection, a top-ranked vulnerability in the 2024 CWE Top 25[11] and 2021 OWASP Top 10[23]—attackers exploit its flexible payload construction to craft evasive variants, repeatedly testing WAF defenses until successful injection. This underscores SQL injection–based adversarial attacks as critical for evaluating WAF capabilities, providing early warnings to refine detection rules and strategies, and mitigating risks like data breaches.

Current WAF detection mechanisms fall into two primary categories [7]: rule-based methods rely on pattern matching but struggle with SQL injection's evolving mutations, suffering false positives (overly rigid rules) and false negatives (obfuscated payloads evading predefined patterns) [27, 29, 35]; machine learning–based methods,

Authors' Contact Information: Bing Zhang, bingzhang@ysu.edu.cn; Chao Liu, liusuanchao@163.com; Rong Ren, rongrenysu@hotmail.com; Qian Wang, wangqianysu@163.com; Jiadong Ren, jdren@ysu.edu.cn, School of information science and engineering, the Key Laboratory for Software Engineering of Hebei Province, Yanshan University, QinhuangDao, Hebei Province, China.

ACM 1557-7392/2026/1-ART

https://doi.org/10.1145/3788286

trained on large datasets, offer generalization but remain prone to errors due to reliance on high-quality labeled data and limited model interpretability for borderline inputs[8, 9, 17, 22, 39].

For this reason, in cybersecurity defense practices, payload-based penetration testing has become a key method for evaluating WAF defense capabilities. By constructing and delivering actual payloads, researchers can immediately observe whether the WAF blocks or allows them, thus accurately reflecting its defense rules and detection strategies in real-world scenarios. Semantic-preserving mutation of payloads can bypass detection for payloads that would otherwise be detected, thereby increasing the WAF's FNR. The process of turning "detectable payloads into undetectable ones" not only provides a novel and quantifiable approach for assessing WAF robustness but also facilitates the comparison of the practical effectiveness of different mutation strategies.

These payload-based penetration testing methods involve two categories. The first is payload-based WAF testing [1, 5, 6, 34, 41], which involves repeatedly testing SQL injection payloads from a dataset until one successfully bypasses the WAF. However, such approaches typically rely on large-scale, unguided payload generation and testing, resulting in low efficiency, high resource consumption, and limited coverage of complex and diverse attack scenarios. Moreover, SQL injection payloads often contain malicious SQL fragments, special characters (such as *quotes*, *comment symbols*), SQL keywords (such as *UNION*, *SELECT*), and logical operators (such as *AND*, *OR*). With the continuous refinement of WAF detection rules, these statically generated payloads are becoming increasingly easy to detect and block. The second category is mutation-based WAF testing [13, 16, 21, 25, 38], which generates new attack variants by mutating previously blocked payloads through operations such as character substitution, insertion, deletion, reordering, and encoding transformation. This approach not only enhances evasion success rates through optimized mutation strategies but also capitalizes on historical attack patterns from blocked payloads to iteratively produce similar variants. By targeting rule sets or feature signatures that WAFs have previously successfully used to detect payloads, mutation-based testing serves a dual purpose: (1) evaluating the effectiveness of existing WAF rules in recognizing evolving attack patterns, and (2) identifying gaps in rule-based detection mechanisms.

In mutation-based WAF testing, existing methods are still insufficient in several key aspects. SQL injection comprises many different types [40]. However, payloads contained in existing datasets (e.g. [20, 31]), as well as those generated using randgen [13], lack structural and semantic diversity; payloads produced from custom CFG rules [5, 6] suffer similar limitations and cannot adequately cover diverse attack scenarios. Secondly, most works [13, 16, 38] mainly adopt random or simple rule-based mutation strategies, which are inefficient and prone to breaking payload semantics. In addition, simple string-replacement [13, 16, 38] or integer-encoding schemes [13, 16, 25, 38] often generate mutated payloads that become nonfunctional, significantly reducing bypass success rates. Thirdly, random-mutation approaches [13, 25] and reinforcement-learning-based searches [16, 38] incur high computational cost and require many requests, making them unsuitable for black-box or real-time testing scenarios.

To address issues in existing methods, we propose *BWAFSQLi*, a general and extensible attack and defense framework for WAFs. Compared with [13, 16, 25, 38], *BWAFSQLi* proposes a payload generation method based on context-free grammar [10] guided by a *convergence factor*, which covers more types of SQL injection syntax rules, generates a more refined SQLiCFG dataset, and achieves higher semantic equivalence between pre- and post-mutation payloads than [13, 25]. To verify the semantic consistency of the generated payloads, we build an equivalence validation platform, *WebSETest*, based on HTML, PHP, and MySQL. To address the incomplete mutation strategies in [13, 25, 38], we design two new strategies: *Quotation Mark Encoding* and *Comment Extension*, expanding the mutation strategies for SQL injection evasion and improving WAF bypass capability. Each malicious string in a payload can be transformed by multiple mutation strategies (e.g., or → /!*or*/, or → OR), forming a large mutation payload combination space. Exhaustively enumerating all possible combinations incurs significant computational overhead. To address this issue, we propose a mutation-operation selection mechanism jointly driven by a decay factor and a historical data table, combined with a multi-position simultaneous mutation

mechanism to improve mutation efficiency and sample diversity. The decay factor dynamically balances exploration and exploitation at different stages; the historical data table records the ID, usage count, and success count of each mutation operation, from which a weight is computed to guide strategy prioritization; the multi-position simultaneous mutation applies the same strategy concurrently to all applicable positions. This mechanism enables the rapid generation of high-quality, semantically equivalent adversarial samples under a limited budget and effectively mitigates the inefficiencies caused by the complex action spaces in reinforcement learning [16, 38] as well as the use of random mutations in traditional strategies [13, 25], thereby significantly improving search efficiency and bypass capability in large-scale payload spaces.

Finally, we performed a comprehensive evaluation of *BWAFSQLi* on three datasets (SQLiCFG, HPD [20], and SIK [31]). The evaluation covered three deep-learning-based gray-box WAFs (CNN [17], LSTM [9], and GRU [22]) and eight open-source or commercial black-box WAFs (ModSecurity, Chaitin Safeline, Huawei Cloud, Tianyi Cloud, Cloudflare, AWS, F5 and Fortinet). Results show that *BWAFSQLi* increases the FNR by up to **93.39%** on gray-box WAFs and up to **58.49%** on black-box WAFs, while substantially reducing the number of required HTTP requests. *BWAFSQLi* outperforms three state-of-the-art baselines (WAM [13], DQNSQLi [38], and AdvSQLi [25]) in both bypass effectiveness and efficiency. Additionally, we propose seven payload pre-processing defense strategies that reduce the FNR increase of the eight black-box WAFs by **74.59%** on average. More specifically, when analyzed across different request types—including GET, POST, GET(JSON), and POST(JSON)—the *IFNR* exhibited an average reduction of at least **66.27%**.

In general, this study makes the following contributions:

(1) We propose *BWAFSQLi*, a general and extensible framework for cross-scenario WAF adversarial evaluation, which achieves maximal FNR elevation while minimizing average HTTP requests, as empirically validated across 11 WAFs (3 gray-box, 8 black-box).
(2) We design a convergence-factor-guided context-free grammar for payload generation, significantly enhancing attack pattern quality and diversity while preserving malicious functionality. The resulting *SQLiCFG*[1] dataset is generated.
(3) We extend 13 known mutation strategies by introducing two novel techniques—*Quotation Mark Encoding* and *Comment Extension*—to substantially improve mutation effectiveness.
(4) We develop an automated semantic equivalence verification platform (*WebSETest*[1]) to guarantee functional consistency before and after mutation.
(5) We propose a novel mutation strategy selection mechanism that integrates a *decay factor* and *historical data table*, combined with multi-position mutation, to reduce average HTTP requests and ineffective trials.
(6) Finally, we design seven preprocessing defense strategies tailored to diverse payloads, which completely suppress the FNR increase for WAFs during evaluation—highlighting their practical value.

The rest of this paper is as follows: Section 2 defines the concept of the WAF Injection Bypass; Section 3 designs and implements the *BWAFSQLi* framework; Section 4 conducts experimental evaluation and analysis of *BWAFSQLi*; Section 5 compares *BWAFSQLi* with related work; Section 6 discusses the advantages and limitations of *BWAFSQLi*; Section 7 summarizes the main content of this paper and looks ahead to future research directions.

2 WAF Injection Bypass

WAF injection bypass occurs when an attacker carefully crafts malicious SQL payloads to evade a WAF's detection and successfully carry out an SQL injection. For example, an attacker might send input like *user?id=1' or '1'='1'–+* to exploit a vulnerable query. If the WAF does not properly filter input or its protections are weak, such a request can bypass validation and allow the attacker to extract all user data from the database.

[1]https://github.com/KLSEHB/BWAFSQLi

A WAF is typically deployed at the front end of the system and inspects all incoming requests first (see Figure 1(a)). In this setup, the WAF detects and blocks malicious requests and only lets legitimate traffic through, preventing the attack. However, if the attacker mutates the payload by changing the query structure or using special encodings (for example, *0x1'/**/Or\n'a'='a'–+*), the altered request may evade the WAF. As shown in Figure 1(b), the mutated request can reach the web application and trigger the attack, causing potential data breaches.

Therefore, to bypass a WAF effectively, an attack payload must satisfy both conditions (1) and (2) simultaneously.

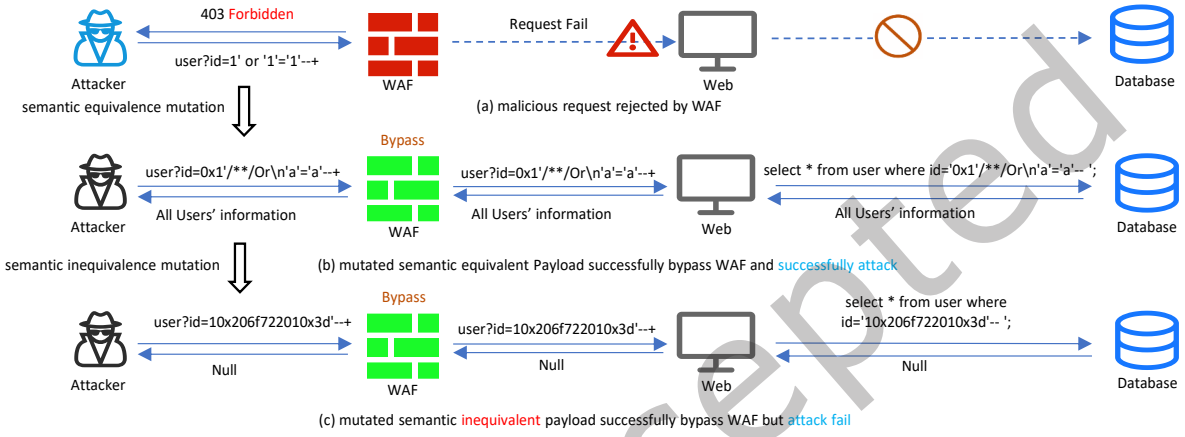

Fig. 1. The attacker used SQL injection to bypass the WAF process.

(1) Mutate blocked SQL injection payloads. WAF rule sets usually rely on known attack signatures. By mutating payloads that a WAF already blocks, an attacker may escape those signature checks. However, generating many new mutations can create invalid or ineffective attacks (for example, syntax errors), which raises cost and lowers success rate. Because WAFs do not always cover every variation of known attacks, transforming payloads with techniques like *Quotation Mark Encoding* or *Comment Extension* often works better to evade detection.

(2) Mutations must maintain semantic equivalence: After mutation, the payload must still perform the same malicious action, return the same data or produce the same database error; otherwise the attack can fail. If a mutation changes semantics (for example, by misplacing comment symbols and terminating the query early), it will not work. Figure 1(c) shows this: mutating *1' or '1'='1'–+* into the semantically different 10x206f722010x3d'–+ bypasses the WAF but fails to retrieve all user data. By contrast, Figure 1(b) shows a semantically equivalent mutation: *0x1'/**/Or\n'a'='a'–+* both evades detection and still extracts the intended data. Some WAFs also monitor query execution time and result set changes, so keeping semantics the same helps avoid behavioral differences that would reveal the attack. Formally, we express semantic equivalence as:

$$\text{Semantics}(P) = \text{Semantics}(P'), \tag{1}$$

where P denotes the original payload, $P' = mutate(P)$, and $Semantics(P)$ denotes the semantics of P in the target system.

(3) Bypass the WAF under realistic threat models. Real WAFs may use pattern matching or machine learning, but vendors rarely disclose internal algorithms or model parameters. We therefore construct bypass scenarios under strict assumptions [14].

- Gray-box WAFs [37]: A gray-box WAF returns a confidence score $p_y \in [0, 1]$ for each input (common in deep-learning WAFs). An attacker studies how the WAF response score $WAF\_Response(P')$ for the mutated payload P' relates to a threshold t. If $WAF\_Response(P') < t$, the attacker treats the payload as potentially bypassing detection.
- Black-box WAFs [3, 36]: A black-box WAF returns only a binary label $y \in \{0, 1\}$. This is the strictest, most realistic setting for WAF-as-a-Service. Attackers can only observe whether the classification changed: $WAF\_Response(P) \neq WAF\_Response(P')$.

In both settings, the attacker uses an iterative optimization strategy. They use the limited feedback from the WAF to adjust mutation combinations step by step, converging toward payloads that successfully bypass detection.

3 BWAFSQLi Design and Implementation

BWAFSQLi is designed to perform semantic equivalent mutations on SQL injection payloads for the purpose of WAF attack simulation and evaluation. This process helps WAF vendors identify potential vulnerabilities in advance, issue early warnings, and improve their systems through targeted optimization. The *BWAFSQLi* framework consists of six modules: *SQL Injection Payloads Generation* (Section 3.1), *Mutation Payloads Generator (MPG)* [*Adversarial Payload Tokenization* (Section 3.2), *Token Semantic Equivalence Mutation* (Section 3.3), *Mutation Strategy Selection* (Section 3.4), *Semantic Equivalence Assessment* (Section 4.2)], and *WAF Attack and Defense* (Section 3.5). The overall workflow is illustrated in Figure 2.

3.1 SQL injection Payloads Generation

In SQL injection attacks, attackers usually insert malicious input (such as *1 or 1=1*) into a URL or a web form and submit it to the backend server. The server extracts the parameter, appends it to a fixed SQL query, executes the resulting statement, and triggers the injection. As Figure 3 shows, the attack focuses on placing malicious code in the dynamic part of the query, such as *$_GET['id']*.

Many public datasets for SQL injection payload generation contain issues. For example, SIK [31] includes hardcoded SQL fragments inside full payloads (see the first payload in Figure 3). Those hardcoded parts can confuse classification models. HPD [20] even labels strings like *5739-5839* and *1wwis* as malicious payloads, which introduces labeling errors.

Furthermore, existing SQL injection datasets often lack coverage of the full diversity and complexity of real attacks. For example, Appelt et al. [5, 6] used context-free grammars to generate only six categories of SQL injection payloads based on character types and injection techniques. To address limitations in current datasets, such as limited diversity, labeling errors, and structural inconsistencies, we propose a context-free-grammar payload generator guided by a convergence factor.

Concretely, we define 58 generation rules that cover 18 SQL injection types, including stacked injection, UNION injection, tautology injection, error-based injection, boolean-based blind injection, and time-based blind injection. We generate these types in both string and numeric forms and include three benign types for balance (see Table 1). These rules produce a more comprehensive and better structured SQL injection payload dataset.

$$SQLiCFG = \{P_1, \ldots, P_i, \ldots, B_1, \ldots, B_i, \ldots\}, \tag{2}$$

Where P_i denotes a malicious payload and B_i denotes a benign payload. Depending on the needs, we can flexibly choose which SQL injection types to generate.

Table 1 lists the 18 malicious payload types and 3 benign types that *BWAFSQLi* can produce. Compared with the six types in Appelt [5, 6], our method covers many more attack types. The full context-free grammar definition is provided in Table A1 in the Appendix.

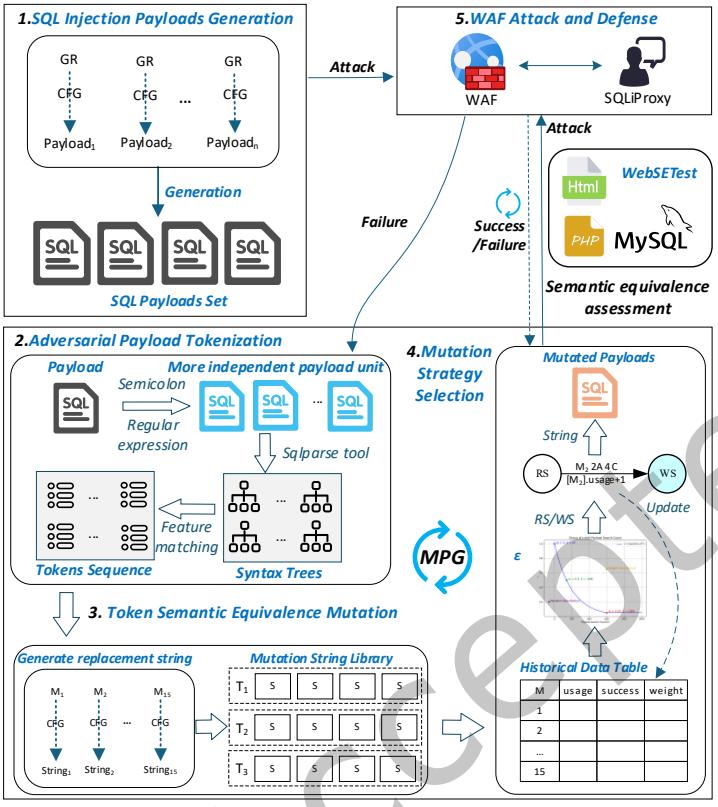

Fig. 2. The workflow of *BWAFSQLi*.

Hardcoded Sections Logical Sections

SELECT * FROM users WHERE id = $_GET['id']

1 KAG	select * from users where id = 1 or "{%" or 1 = 1 -- 1
2 HPD	5739-5738
3 HPD	1wwis

Fig. 3. Examples of SQl injection payloads in different Datasets.

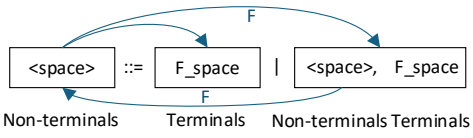

Fig. 4. An example of context-free grammar rules.

Table 1. SQL injection payloads and benign payloads classified by character type and injection method.

Character type	**Example**	**Can be generated by**
numeric	1 or 1=1#	Appelt et al. [5, 6], BWAFSQLi
single quotation mark	1' or 1=1#	Appelt et al. [5, 6], BWAFSQLi
double quotation mark	1" or 1=1#	Appelt et al. [5, 6], BWAFSQLi
single quotation mark with multiple parentheses	1') or 1=1#	BWAFSQLi
double quotation mark with multiple parentheses	1") or 1=1#	BWAFSQLi
single quotation mark with percent sign	1%' or 1=1#	BWAFSQLi
double quotation mark with percent sign	1%" or 1=1#	BWAFSQLi
Injection method	**Example(numeric)**	**Can be generated by**
union	1 union select 1,2,3	Appelt et al. [5, 6], BWAFSQLi
boolean-based blind	1 or substr(user(),1,1)='r'	Appelt et al. [5, 6], BWAFSQLi
tautology	1 or 'JT' = 'JT', 1 or 87 like (select 87)	BWAFSQLi
error-based blind	1 or updatexml(1,concat(0x7e,(select database()),0x7e),0)	BWAFSQLi
time-based blind	1 and benchmark(1000000,rand())	Appelt et al. [5, 6], BWAFSQLi
Stacked payloads	**Example**	**Can be generated by**
insert data	1; insert into users SET password=16;	BWAFSQLi
update data	1; update users (name) VALUES ('tom') where id=1;	BWAFSQLi
delete table	1; drop table users;	BWAFSQLi
delete data	1; delete from users where id=1;	BWAFSQLi
create data	1; create table users;	BWAFSQLi
query data	1; select 0;	BWAFSQLi
Benign payloads	**Example**	**Can be generated by**
word	tar, magnet, landings,...	BWAFSQLi
number	0,1,2,...	BWAFSQLi
MySQL built-in functions	SELECT, INSERT, UPDATE,...	BWAFSQLi

As shown in Figure 4, *[<space>::=F_space | <space>, F_space]* describes a grammar rule, where *<space>* is a non-terminals, *F_space* is a terminals and a production rule, representing a function that can generate a space character. *<space>* and *F_space* form a production rule. According to this rule, *<space>* can be expanded in two ways: the first is to expand into production rule 1 *[F_space]*; the second is to expand into production rule 2 *[<space>, F_space]*, where *<space>* can be further expanded. Through this recursive process, strings composed of multiple spaces can be generated. However, production rule 2 *[<space>, F_space]* includes the non-terminals *<space>*, which means this rule may cause infinite recursion, resulting in the generation of an infinite number of spaces. To effectively address this issue, we propose a payload generation method based on context-free grammar guided by a *convergence factor* to avoid recursive loops and ensure the generation process terminates smoothly. Specifically, we gradually reduce the probability of a rule being selected by setting the *convergence factor F*. Assuming the convergence factor F is 0.5, in Figure 4, at the initial stage, the probabilities of selecting production rules 1 and 2 are both 1. As the iteration proceeds, the probability of selecting production rule 2 *[<space>, F_space]* for the third time is $0.5^2 = 0.25$. Therefore, in the next non-terminals *<space>* generation selection, since production rule 2 is already $0.25 < 1$, production rule 1 *[F_space]* is selected, thereby ensuring that the generation process can terminate smoothly. Based on context-free grammar and the classification of SQL injection payloads and benign payloads in Table 1, we formulated 58 payload generation rules (GR) in Tables A2, A3 and A4 in the Appendix.

Figure 5 illustrates the generation process of the SQL injection payload *1' or '1'='1';drop table users−+* based on Rule 1 (*[SQLInjection_Payload ::= <NumericContext> | <QuoteContext> | <ParenthesesContext> | <PercentContext>]*). In the example, the initial production for Rule 1 is chosen as *<QuoteContext>* (Rule 3). Expanding Rule 3 triggers a chain of further expansions: Rule 3 invokes Rules 10, 12, 6, 7 and 11; Rule 6 invokes Rule 17; Rule 7 invokes Rule 8; Rule 17 invokes Rules 12 and 18; Rule 8 invokes Rule 26; and Rule 26 invokes Rule 12. Through repeated application of these productions together with the convergence factor F, the derivation ultimately yields the terminal sequence *[F_Digit, opSquote, F_space, opOr, F_space, F_tautology_string, opSem, opDrop, F_space, opTable, F_space, F_TableName, dcmt, opAdd]*.Each terminal maps to concrete text as follows: *F_Digit* → *1*, *opSquote* → *'*,

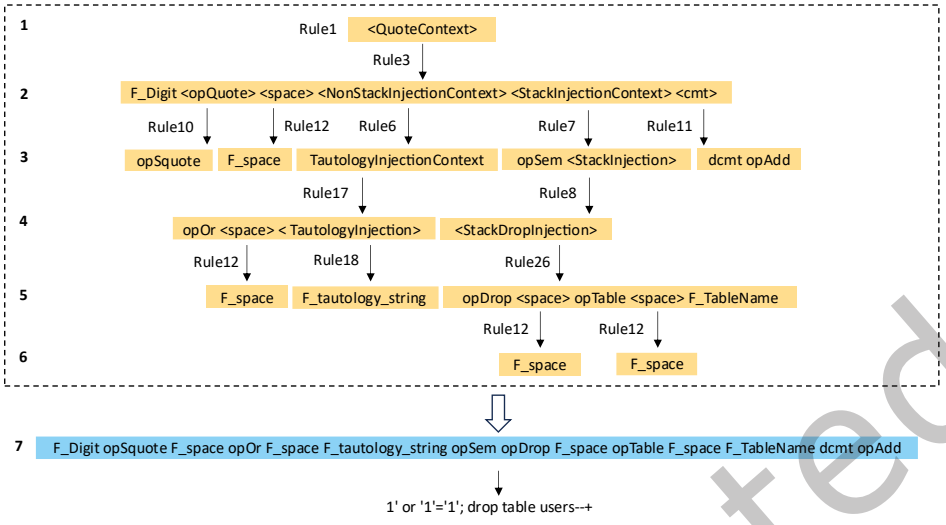

Fig. 5. An example of SQL injection payload generation by generation rules.

F_space → (space), *opOr* → *or*, *F_tautology_string* → *'1'='1'*, *opSem* → *;*, *opDrop* → *drop*, *opTable* → *table*, and *F_TableName* → *users*. Combining these terminals produces the final payload: *1' or '1'='1';drop table users−+*.

3.2 Adversarial Payload Tokenization

The generated attack payloads consist of syntactic elements such as characters, whitespace, operators, and SQL keywords. Although these elements are independent, they work together to change the meaning of an SQL statement. For example, the classic payload *1' or 1=1−+* combines several basic syntactic units to form an effective attack query. This structure makes SQL injection payloads highly decomposable and flexible.

In *BWAFSQLi*, the token operation first parses a payload into several syntactic tokens (e.g., string literals, identifiers, numeric literals, quote markers, comment fragments, operators, and encoded segments); each token type is then mapped to one or more applicable mutation operations, and multiple replacement strings are generated per token according to the selected operations; finally, the replaced tokens are reassembled into a complete payload for subsequent testing or attacks, thereby enabling WAF bypass. When a WAF detects and blocks a payload, attackers can use that payload as a seed to generate semantically equivalent variants. By changing the payload's form without altering its malicious behavior, they can create diverse versions that still bypass static, rule-based detectors. We apply the Sqlparse [2] tool to perform deep syntactic analysis of SQL injection payloads. Sqlparse identifies comment symbols, string literals, keywords, operators, and other components, and clarifies each part's semantic and functional role. This analysis forms the basis for mutation strategies that preserve semantic equivalence. By keeping the payload's core logic intact, for example the truth-preserving expression *or 1=1*, and only altering surface details such as encoding or syntactic patterns, the mutated payload can evade WAF detection while still performing the intended attack.

For numeric SQL injection payloads, such as *1 or 1=1−+*, we can parse them directly with the Sqlparse [2] tool to produce a syntax tree. However, when a payload contains special characters, for example single or double quotes shown in Table 1, Sqlparse may fail to parse quoted segments correctly and break the payload's semantic equivalence. Figure 6 highlights this problem. Parsing *1' or '1'='1';* can yield incorrect segments like *' or '* and *'=',*

which lead to a wrong *Tokens Sequence*. As a result, the mutated payload 10x206f722010x3d'; is not semantically equivalent to the original *1' or '1'='1';*.

To prevent these parsing errors, we apply a regular expression called *MatchRegRule* before parsing with Sqlparse [2]. This preprocessing step normalizes or protects quoted segments so that Sqlparse produces a correct syntax tree and the subsequent mutations preserve semantic equivalence.

$$MatchRegRule = \{\text{^}[-]?[a\text{-}zA\text{-}Z0\text{-}9]*[\%]?[\backslash'"][\backslash)]*\}. \tag{3}$$

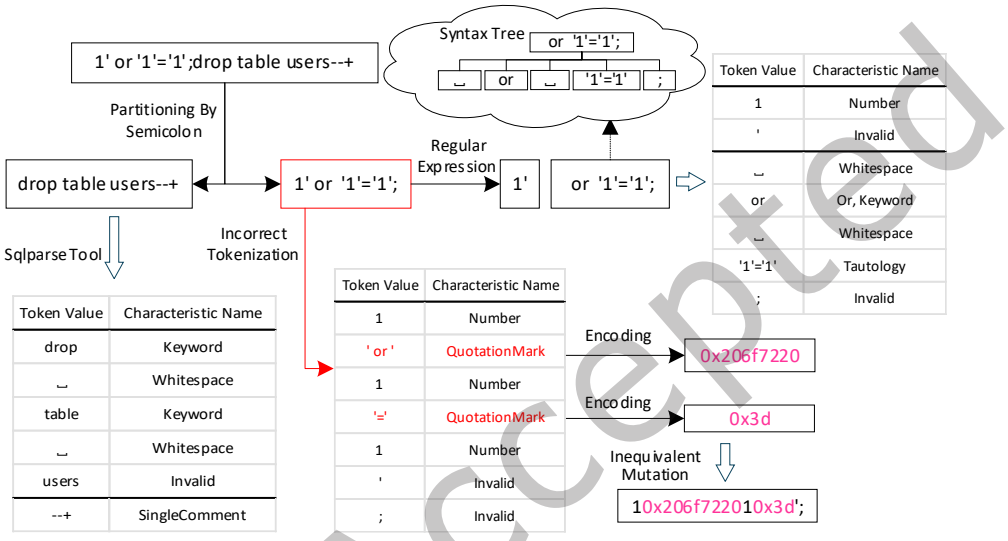

Fig. 6. An example demonstrating the tokenization of a payload into discrete tokens.

MatchRegRule matches strings that may start with an optional -, optional alphanumeric characters, and an optional %, followed by a single quote ' or double quote ", and then any number of *)*. *MatchRegRule* finds the position of the first single or double quote and splits the payload into two parts: the part before the quote (including the quote) and the part after the quote. This prevents quote-closure problems during mutation and helps preserve semantic equivalence between the original and the mutated payload.

For stacked SQL injection payloads such as *1' or '1'='1';drop table users−+*, we first split the payload by the semicolon ";" into segments U. We then apply the quoted-segment division to each segment U_i to obtain $U_i = [U_{i1}, \ldots, U_{ij}, \ldots]$, where each U_{ij} is a more independent unit, for example *1'* or *−+*. A payload P can then be expressed as a sequence of these U_i.

$$P = \{U_1, \ldots, U_i, \ldots\}. \tag{4}$$

We use the *Sqlparse* [2] tool to parse the independent units U_i in P and produce the corresponding syntax tree (see the syntax tree in Figure 6). The leaf nodes (U_{ij}) in the syntax tree fall into two main types: *sqlparse.sql.Token* and *sqlparse.sql.Comparison*, both provided by the *Sqlparse* library. Based on each leaf node's type and value, we define 12 features, as shown in Table 2, which illustrates the mapping relationships among the Characteristic Name, Class Name, Token Types, Match Character, and Mutation Strategies (Table 3), along with corresponding examples.

The specific rules are as follows:

Table 2. The features of tokens and the mapping relationships between token feature types and their corresponding mutation strategies.

Characteristic Name	Class Name	Token Types	Match Character	Example	Mutation Strategies
Tautology	sqlparse.sql.Comparison	None	-	1=1	Tautology Substitution
Whitespace	sqlparse.sql.Token	Token.Text.Whitespace	-	␣	Whitespace to Control Character, Comment Injection in Whitespace
Keyword	sqlparse.sql.Token	Token.Keyword, Token.Keyword.DML, Token.Keyword.DDL,Token.Keyword.CTE	-	select, or	Keyword Case Swapping, Keyword Inline Comment
Number	sqlparse.sql.Token	Token.Literal.Number.Integer	-	1, 0	Integer Encoding
SingleComment	sqlparse.sql.Token	Token.Comment.Single, Token.Comment.Single.Hint	-	−+, −	Comment Extension
MultilineComment	sqlparse.sql.Token	Token.Comment.Multiline, Token.Comment.Multiline.Hint	-	/*comment*/	Comment Rewriting
QuotationMark	sqlparse.sql.Token	Token.Literal.String.Single, Token.Literal.String.Symbol	-	uname, passwd	Quotation Mark Encoding
EqualSign	sqlparse.sql.Token	-	=	-	Equal Swapping
Where	sqlparse.sql.Token	-	where	-	Where Rewriting
And	sqlparse.sql.Token	-	and	-	And Logical Invariant, And Substitution
Or	sqlparse.sql.Token	-	or	-	Or Logical Invariant, Or Substitution
Invalid	-	-	-	;, ’, ”	-

(1) if *Class Name = sqlparse.sql.Comparison*, then the node feature is *Tautology* (true statement).
(2) if *Class Name = sqlparse.sql.Token*, then further match the characteristics based on *Match Character* or *Token Types* to obtain the corresponding *Characteristic Name*.
(3) If none of the above conditions are met, mark it as *Invalid*, indicating that the node (i.e., the *token* with the feature *Characteristic Name = Invalid*) cannot undergo any mutation (such as semicolons, commas, quotation marks, which do not require mutation processing).

Next, we organize each leaf node of the grammar tree into feature groups in the form of (*token value*, *Characteristic Name*), i.e., $[(tv_1, CN_1), \ldots, (tv_i, CN_i), \ldots]$, denoted as D_i (i.e., $U_i \rightarrow D_i$). Finally, all units U_i are parsed into the corresponding feature list D_i, resulting in the *Tokens Sequence*:

$$Tokens\ Sequence = \{D_1, \ldots, D_i, \ldots\} \tag{5}$$

Among them, $D_j = [(tv_1, CN_1), \ldots, (tv_i, CN_i), \ldots]$,contains the feature representations of all leaf nodes in U_i.

3.3 Token Semantic Equivalence Mutation

After extracting the *Tokens Sequence* of a payload that WAF rules blocked, we apply semantically equivalent mutations to the *token values* to produce new variants. These variants keep the original attack semantics but change the payload structure. Such mutations can bypass detectors that rely on string matching or fixed syntactic patterns and thus increase evasion success rates. Because we start from previously blocked samples, this method targets the matching boundaries of existing WAF rules. If a mutated variant bypasses detection, it shows the original rule does not cover some semantically equivalent forms and exposes a weakness in the WAF. If a payload bypasses a WAF on the first try, that usually means the WAF's rules for that attack pattern are missing or insufficient, and vendors should be notified to update their rules or add defenses.

In this section, we extend the 13 token value mutation strategies from Qu et al. [25], Wang et al. [38], and Demetrio et al. [13]. These include techniques such as *Keyword Case Swapping*, *Whitespace to Control Character*, and *Comment Injection in Whitespace*. We also introduce two new strategies not covered by prior work: *Quotation Mark Encoding* and *Comment Extension*. *Quotation Mark Encoding* converts quotation marks and their inner content into hexadecimal (for example, *'user' ⟶ 0x75736572*) to evade quote and keyword detection. *Comment Extension* appends extra text to a trailing comment (for example, *1' or 1=1# ⟶ 1' or 1=1#hello*) to disrupt WAF detection logic and improve bypass rates. Table 3 lists these mutation strategies and gives example transformations.

It is important to note that for the *Integer Encoding* mutation strategy in Table 3, if a payload contains critical functions that require strict input parameters (Table 4), especially parameters that must be integers, this mutation

Table 3. Token Mutation Strategies: A Comparative Analysis.

Number	Mutation Strategies	BWAFSQLi	Qu et al. [25]	Wang et al. [38]	Demetrio et al. [13]	Examples
M1	Keyword Case Swapping					select 1,2,3 → SeLeCt 1,2,3
M2	Whitespace to Control Character					or 1 = 1 → or\t1 = 1
M3	Comment Rewriting					or 1/*hello*/= 1 → or/*hi*/1 = 1
M4	Comment Extension		×	×	×	or 1 = 1–+ → or 1 = 1–+hello or 1 = 1# → or 1 = 1#hello
M5	Integer Encoding					1 or 1 = 1 → 0x1 or 1=1
M6	Equal Swapping					1 or 1 = 2 → 1 or 1 like 2
M7	And Logical Invariant					and table_name="users" → and True and table_name="users"
M8	Or Logical Invariant					or 1 = 1 → or False or 1 = 1
M9	Comment Injection in Whitespace					or 1 = 1 → or/*hi*/1 = 1
M10	And Substitution					1 or 1 = 1 → 1 || 1 = 1
M11	Or Substitution					and table_name="users" → && table_name="users"
M12	Keyword Inline Comment			×	×	select 1,2,3 → /!*select*/ 1,2,3
M13	Where Rewriting			×	×	where table_name="users" → where False or table_name="users" where table_name="users" → where True and table_name="users"
M14	Tautology Substitution			×	×	1 = 1 → (select 7) like 7 1 = 1 → '2@HJ{'='2@HJ{' 1 = 1 → (select 1) like (select if(user() = 'root', 1, 0)) 1 = 1 → 2<>3 and (select 1)
M15	Quotation Mark Encoding		×	×	×	table_name="users" → table_name=0x7573657273

may break the function or cause execution to fail. For example, mutating *sleep(1)* to *sleep(0x1)* prevents correct execution because *sleep()* requires an integer parameter. To avoid such problems, we first check whether a number is used as a parameter in a sensitive function before applying *Integer Encoding*. If it is, we skip this mutation to maintain the function's behavior and preserve semantic equivalence.

Table 4. SQL functions that use integer-type as parameters.

SQL functions	Usage	Description
SLEEP()	SLEEP(seconds)	pause execution for the specified number of seconds.
BENCHMARK()	BENCHMARK(count, expression)	Execute the given expression the specified number of times for performance testing.
LEFT()	LEFT(string, length)	Return a substring of specified length from the left side of the string.
SUBSTR()	SUBSTR(string, start, length)	Starting from a specified position, extract a substring of a specified length.
ELT()	ELT(index, string1, string2, ...)	Return the string selected based on the index.

To ensure semantic equivalence before and after token mutation and to increase the scalability, diversity, and complexity of mutations, we define 26 payload mutation rules (MR), presented in Tables A5, A6, and A7 of the Appendix. These rules are based on context-free grammars and implement the 15 mutation strategies listed in Table 3. They let us compose and flexibly generate semantically equivalent *token values*.

Based on the *Tokens Sequence* generated by the payload, denoted as $D_j = [(tv_1, CN_1), \ldots, (tv_j, CN_j), \ldots]$, we select a mutation strategy M_k and a context-free grammar rule for each (tv_j, CN_j), where CN_j is the *Characteristic Name*. We then combine these selections to create a new mutation feature list $token_j$, as follows:

$$(tv_j, CN_j) \rightarrow token_j = \{tv_j, CN_j, (M_1, R_1) \mid \ldots \mid (M_k, R_k) \ldots\}. \tag{6}$$

After generating a new list of mutation features for all (tv_j, CN_j) in D_j, record it as T_j(*i.e.*, $D_j \rightarrow T_j$):

$$T_j = \left[(tv_1, CN_1, (M_1, R_1) \mid \ldots \mid (M_k, R_k) \ldots), \ldots, \left(tv_j, CN_j, (M_1, R_1) \mid \ldots \mid (M_k, R_k) \ldots\right), \ldots\right], \tag{7}$$

where R_k denotes the result set $\{r_1, \ldots, r_j, \ldots\}$ obtained after tv_j is mutated using the M_k mutation strategy. Finally, the *Mutation String Library* is obtained:

$$Mutation\ String\ Library = \left\{T_1, \ldots, T_j, \ldots\right\}. \tag{8}$$

As shown in Figure 7, the token *or* has Token Types *Token.Keyword* and *Match Character or*. Using the mapping of Characteristic Name, Class Name, Token Types, and Match Character in Table 2, we identify the *Characteristic*

Name of *or* as *Keyword* and *Or*. Based on these values, we can apply four mutation strategies from Table A5: *Keyword Case Swapping (M_1)*, *Or Logical Invariant (M_8)*, *Keyword Inline Comment (M_{12})*, and *Or Substitution (M_{11})*. We then generate multiple mutable, semantically equivalent strings according to the grammar rules of the selected mutation strategies.

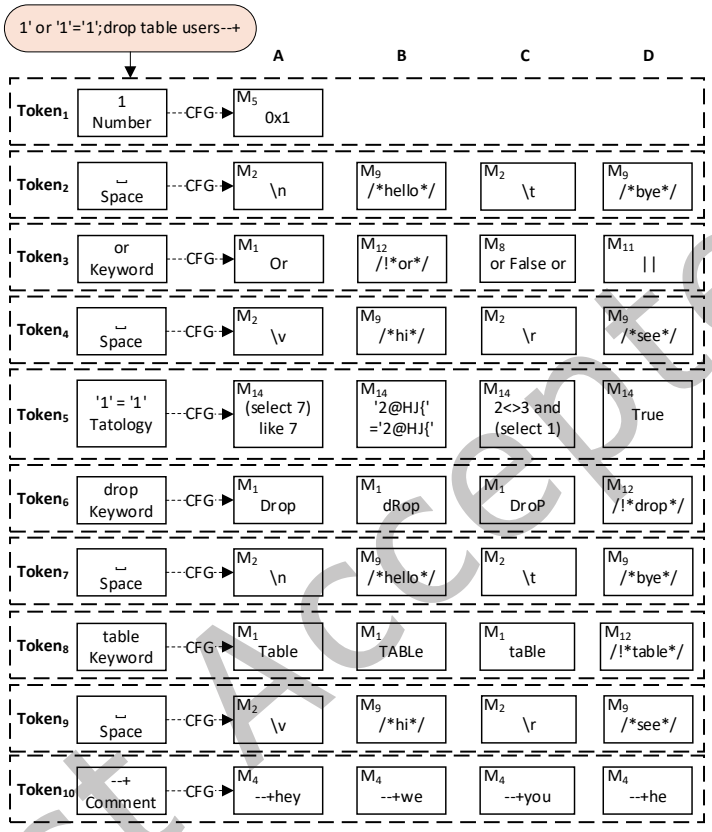

Fig. 7. An example of *Mutation String Library* generation.

3.4 Mutation Strategy Selection

From the *Mutation String Library* $= \{T_1, \ldots, T_j, \ldots\}$ in section 3.3, we see that each token can undergo multiple mutation strategies (such as *Keyword Case Swapping* or *Quotation Mark Encoding*), and each strategy can produce multiple strings (for example, *Keyword Case Swapping* changes *or*⟶*Or* or *or*⟶*OR*). Traversing all possible combinations would require enormous system resources. Methods based on reinforcement learning [16, 38] have high computational complexity and long training times, while random mutation strategies guided by malicious scores [13, 25] select strategies randomly, which may be inefficient, increase malicious scores, and waste resources. To overcome these issues, we propose a mutation strategy selection method driven jointly by a *decay factor* and a *historical data table*, and we introduce a *multi-position* simultaneous mutation mechanism.

3.4.1 Mutation Strategy Selection Process.
Through *Mutation String Library*, we can determine that the mutation strategy set for all tokens in the current attack payload is *Mutation Strategies* $(MS) = \{M_1, \ldots, M_i, \ldots\}$. The principle behind selecting mutation strategies to mutate the tokens in the original payload and generate new payloads for WAF attack testing is as follows:

(1) Mutation strategy selection process: Mutation strategy M_x is selected from the set *MS*. Based on the *multi-position* simultaneous mutation mechanism, all tokens containing M_x (such as $token_p = \{tv_p, CN_p, (M_x, R_a)|(M_y, R_b)\}$, $token_q = \{tv_q, CN_q, (M_x, R_c)|(M_y, R_d)\}$) are processed. A candidate replacement string is selected from each corresponding set R_a and R_c ($R_a.r_i, R_c.r_j$, where $R_a.r_i \neq R_c.r_j$) to replace the original token values. A new SQL injection payload is then generated and used for WAF bypass testing.
(2) *Budget* process definition: If the payload fails to bypass the WAF, another unused strategy M_y from the set *MS* is selected and the mutation process is repeated. At most n mutation strategies (where $n \leq len(MS)$) can be used in a single attempt. The process terminates once one of the following conditions is met: (i) the number of selected strategies reaches n; (ii) all strategies in *MS* have been attempted; (iii) the mutated payload successfully bypasses the WAF. This sequence is defined as a *Budget* process.
(3) *Step* process definition: Some mutation strategies, such as M_x, can produce multiple mutation strings. For example, $R_a = \{r_1, \ldots, r_i, \ldots\}$, but a single *Budget* process usually only covers one of them. To increase coverage, the *Budget* process is repeated n times to form a *Step* process. A *Step* process may be repeated up to m times. It terminates when any of the following conditions is met:(i) the number of executed *Budget* processes reaches the predefined upper limit n; (ii) all candidate string sets R in the *Mutation String Library* have been covered; (iii) a mutated payload successfully bypasses the WAF.

3.4.2 Mutation Strategy Selection Standard.
In the *Step* process described above, $m * n$ mutation strategy selections are involved. Theoretically, as shown in Figure 8, the exhaustive method would require at least *2*5*5*5*5*5 -1 = 6249* attempts to achieve full coverage, which is very inefficient. To reduce the large search space in traditional methods, we first define a threshold ε, which gradually decreases based on a *decay factor* D and the number of payloads detected by the WAF, C, until it reaches a minimum value *min_ε*. The formula for ε is as follows:

$$\varepsilon = \max\left(\min\_\varepsilon, D^C\right). \tag{9}$$

Next, we generate a random number *rd* between $[0, 1]$ and compare it with ε. If $rd < \varepsilon$, we randomly select a mutation strategy from the set *MS* (Random Selection, *RS*). If $rd \geq \varepsilon$, we use a weighted selection (*WS*) mechanism, where the mutation strategy is chosen based on historical data recorded in the *Historical Data Table*. This table tracks each strategy's ID, the number of times it has been used, the number of successful bypasses, and the computed weight W_i. We calculate the weight as follows:

$$W_i = M_i\_success \times \left(1 - \frac{M_i\_usage}{\sum_{j=1}^{len(MS)} M_j\_usage}\right). \tag{10}$$

In Equation 10, $M_i\_success$ represents the number of times the strategy has successfully bypassed the WAF, and M_i_usage is the number of times it has been selected during the *Step* process. The denominator sums the total selections of all strategies in *MS*. The weight W_i acts as a global indicator across payloads, allowing previous selection outcomes to guide and optimize future strategy choices. During weighted selection, the strategy in *MS* with the highest unused weight is chosen. If the highest-weight strategy has already been used, selection continues in descending weight order until an unused strategy is found.

As shown in Equation 9, Figure 8, and Equation 10, ε decreases gradually to its minimum value *min_ε*. When $min\_\varepsilon = 0.05$ and $D = 0.995$, calculations show that ε reaches about 0.5 after mutating 138 full payloads and

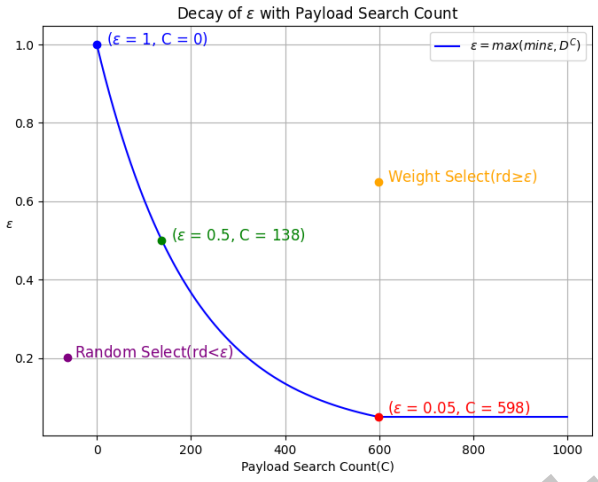

Fig. 8. ε change curve ($min\_\varepsilon = 0.05, D = 0.995, rd \in [0, 1]$).

reaches the minimum after 598 payloads. This means that the strategy selection process is mostly random at the beginning. As the number of WAF-detected payloads C grows, ε decreases, and selection gradually shifts to the weighted approach with only a small chance of random choice. Effectively, ε models a learning process. After observing nearly 600 payload mutations, the mechanism uses the *Historical Data Table* to compute and rank weights for different mutation strategies, enabling more informed and efficient selections in subsequent mutations.

3.4.3 Instances of Mutation Strategy Selection.
Figures 7, 9, and 10 demonstrate two representative cases of the payload mutation process for WAF-bypass failures. Specifically, Figure 9 presents a complete search process for the payload *1' or '1'='1';drop table users−+* (initially featured in Figure 7) during its first mutation attempt. Notably, this initial phase is characterized by an empty *Historical Data Table*, resulting in strategy selection primarily relying on Random Selection (*RS*). In contrast, Figure 10 illustrates the mutation strategy selection for *-100' union select user()#* after multiple optimization iterations. Here, the *Historical Data Table* contains accumulated knowledge from previous mutations, enabling weighted selection (WS) to dominate the selection process through informed historical references.

In initial phase, the *success* (success count) and *usage* (usage count) of all mutation strategies in *Historical Data Table* are set to zero. Then, the Mutation Strategy Selection Process for *1' or '1'='1';drop table users−+* is as follows:

(1) *Mutation String Library* and *MS* Generation: For the payload *1' or '1'='1';drop table users−+*, the *Mutation String Library* is generated (Figure 7), and based on this library, the $MS_C = \{M_1, M_2, M_4, M_5, M_8, M_9, M_{11}, M_{12}, M_{14}\}$ is generated (Figure 9).
(2) Iterative mutation process (Figure 9):
   (a) *Step = 1*, *Budget = 1*: With the initial ε value set to 1, the strategy selection follows a random policy. A mutation strategy is randomly selected from *MS* (such as M_2). The usage count forM_2 is incremented, and all tokens associated with M_2 (such as $Token_2$, $Token_4$, $Token_7$ and $Token_9$) are mutated accordingly (such as replaced with variants in *2A* , *4C*, *7A* and *9C*). A new payload is generated and tested for WAF evasion.
   (b) Subsequent *Steps*: The process of strategy selection, payload mutation, and WAF testing is repeated until *Step* 5 is reached. An effective mutation combination $[M_{12}, M_{14}]$ is ultimately identified. This

transforms the original payload into *1' /!*or*/ True;/!*Drop*/ /*Table*/ users–+*, successfully bypassing the WAF. The success count for M_{14} is updated in the *Historical Data Table* accordingly.

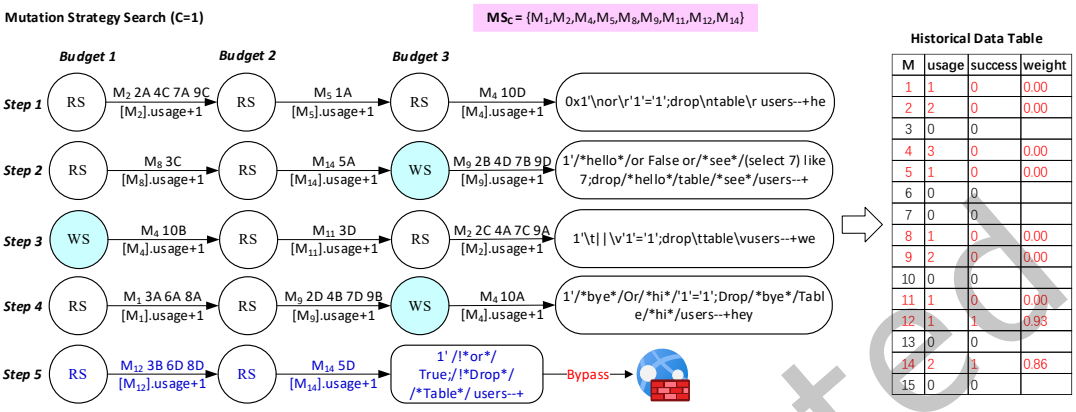

Fig. 9. Search for mutation strategies targeting initially failed WAF payload.

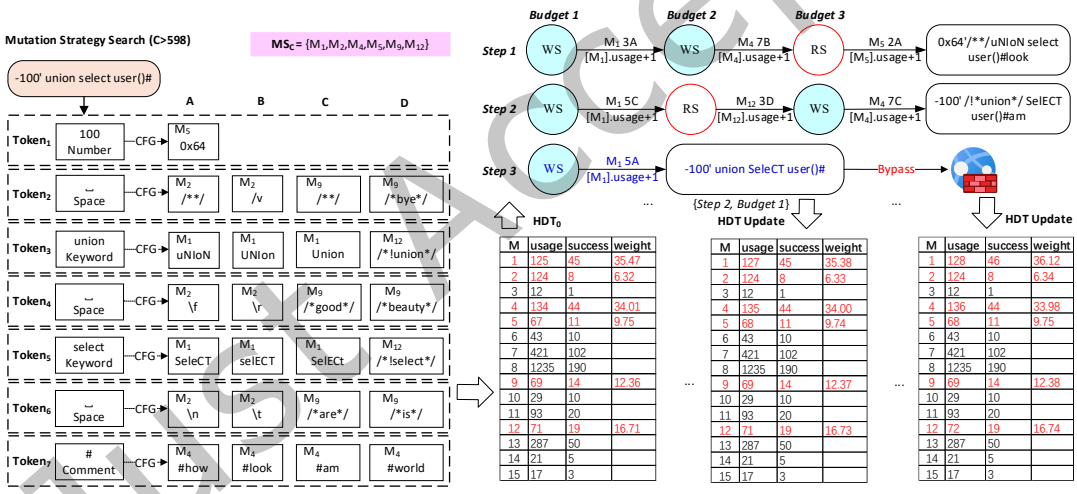

Fig. 10. An illustrative instance of the mutation strategy search process for the *C-th* payload (*C>598*) that fails to bypass the WAF.

As the mutation process progresses, the system accumulates *usage* and *success* data for different mutation strategies. The *Historical Data Table* gradually matures, providing more informative guidance for subsequent strategy selection. The Mutation Strategy Selection Process for *-100' union select user()#* (Firgue 10) is then proceeded below.

(1) At *Step = 1* and *Budget = 1*, the generated random number *rd* is greater than the current ε, so a weight-based selection strategy is used. According to the *Historical Data Table*, the weight of M_1 is calculated as

$W_1 = 45 * (1 - 125/(125 + 124 + 134 + 67 + 69 + 71)) \approx 35.47$, which is the highest, and therefore M_1 is prioritized for mutation.

(2) At *Step = 1* and *Budget = 3*, the generated random number *rd* is very small and less than the current ε, so a random selection strategy is adopted, and M_5 is ultimately chosen for mutation. This mechanism ensures that the selection process can escape local optima, and even when ε is very small, there is still a minimal probability of performing random selection.

After three *Step* iterations, an effective mutation combination $[M_1]$ is identified. The resulting payload *-100' union SeleCT user()#* successfully evades the WAF.

3.5 WAF Attack and Defense

After mutating a SQL injection payload, it must be encapsulated into an HTTP request and sent to the WAF for attack testing. Based on the WAF's response, we assess whether the mutation was successful. To facilitate this process, we designed an intermediate proxy called *SQLProxy* (see Figure 11), which sits between the *Mutation Payloads Generator (MPG)* and the WAF. *SQLProxy* is responsible for wrapping payloads into HTTP requests and analyzing WAF responses to determine bypass success. Additionally, *SQLProxy* supports the integration of pre-processing defense strategies to enhance the robustness of WAF detection.

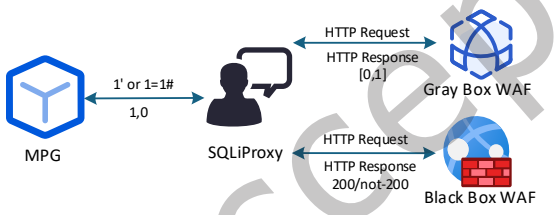

Fig. 11. Overview of the *SQLProxy*.

3.5.1 *WAF Attack Testing.*

(1) Black-box WAF: *SQLProxy* encapsulates payloads into different HTTP request types (GET, POST, GET(JSON), POST(JSON)) to test the black-box WAF. If the WAF detects a malicious request, it returns a non-200 HTTP status code, and *SQLProxy* sends 1 to *MPG*, indicating a failed bypass. If the status code is 200, *SQLProxy* sends 0 to *MPG*, indicating a successful bypass. Different HTTP request types transmit payloads differently, especially GET requests, which usually pass SQL injection payloads as URL parameters. To avoid semantic errors or data loss caused by special URL characters (listed in Table 5), these characters must be properly URL-encoded. As shown by Qu et al. [25], failing to encode special characters in GET requests can break semantic equivalence. For example, the URL *http://localhost?name=1 or/&/1=1#&&pwd=123456* will misinterpret the first *&* and the *#* if sent directly, preventing the correct parsing of parameters. Proper URL encoding ensures all parameters are transmitted accurately and reliably.

(2) Gray-box WAF based on deep learning: In a deep learning-based gray-box WAF, *SQLProxy* acts as a classifier, encapsulating the payload into an HTTP POST request for testing. The gray-box WAF generates a confidence score. If the score is below the threshold *t*, the payload is not identified as an attack, and *SQLProxy* sends 0 to *MPG*, indicating a successful bypass. If the score exceeds *t*, the payload is recognized as a potential attack, and *SQLProxy* sends 1 to *MPG*, indicating a failed bypass.

Table 5. Special characters in URLs.

Special characters	Description	URL encoding
?	used to separate the URL path and query string	%3F
&	used to separate multiple query parameters	%26
=	keys and values used to distribute query parameters	%3D
#	indicates the anchor part of a URL	%23
%	URL-encoded characters	%25
/	URL path separator	%2F
+	URL encoding of space characters	%2B or %20

3.5.2 *WAF Defense.*

After collecting many mutated attack payloads that successfully bypassed WAFs, we performed reverse analysis using the mutation strategies summarized in Table 3. This analysis revealed deeper insights into attack techniques and guided the design of a set of lightweight, strategy-driven preprocessing defenses (see Table 6). We integrate these defenses into *SQLProxy* so it can preprocess SQL injection payloads before they reach the WAF, which significantly reduces the risk of successful injections.

The seven preprocessing strategies we propose (e.g., *Normalize Letter Case*, *Comment Replacement*) are not simplistic input sanitization techniques. Their objective is not to transform malicious payloads into benign ones, but rather to enable WAF rules to effectively identify and block them through reverse analysis of mutation strategies. In other words, the payloads processed by our defensive strategies may still retain malicious intent, but they are modified into forms that can be detected and intercepted by WAF rules. Traditional defense methods such as Prepared Statements, Query Parameterization, and database constraints exhibit significant limitations in this scenario.

- Prepared Statements effectively prevent traditional SQL injections by strictly separating SQL code from user input. However, their deployment requires modifying application logic or SQL code, incurring high costs for legacy systems. *BWAFSQLi*, which does not interact with the target application code, renders this approach inapplicable to the scenario discussed in this paper.
- Query Parameterization mitigates injection risks by binding parameters to fixed SQL templates. Nevertheless, its efficacy depends on consistent implementation—residual string concatenations or third-party bypasses may still be exploitable. *BWAFSQLi* focuses solely on filtering payloads capable of bypassing WAFs, rather than reconstructing the original request logic, making this approach unsuitable for the context of this study.
- Database Constraints (e.g., type checks, triggers) provide only passive defenses at runtime and cannot proactively detect or analyze traffic, thereby offering no improvement to the threat detection capabilities of upstream WAFs. This approach is also inapplicable to the research scenario addressed in this paper.

In contrast, our strategy works at the WAF layer, normalizing, reconstructing, and restoring the semantics of payloads before they reach the firewall. This greatly improves the WAF's ability to detect malicious payloads (see Table 21). The mechanism combines precision and adaptability, providing an efficient, practical enhancement that integrates seamlessly into existing WAF systems without requiring complex engineering changes.

Table 6. Pre-processing defense strategies and examples.

Pre-processing defense strategy	Examples
Normalize Letter Case	SeLeCt 1,2,3 → select 1,2,3
Comment Replacement	1 or/*hello*/1=1 → 1 or 1=1
Logical Operator Replacement	1 || 1=1 → 1 or 1=1
Control Character Conversion	1/n or/f1=1 → 1 or 1=1
Inline Comment Replacement	1 /*!or*/ 1=1 → 1 or 1=1
Hexadecimal to Character Conversion	Table_name=0x7573657273 → table_name="users"
Hexadecimal to Number Conversion	0x1 or 1=1 → 1 or 1=1

4 Evaluation Results

We evaluate the semantic equivalence and performance of *BWAFSQLi*, primarily addressing the following key questions:

- RQ1 in Section 4.2: Can *BWAFSQLi* ensure semantic equivalence between the original and mutated SQL injection payloads?
  To evaluate the ability of *BWAFSQLi* to maintain semantic equivalence before and after payload mutation, we conducted experiments on a custom-designed semantic equivalence verification platform—*WebSETest*. Specifically, we compared the execution results of mutated payloads generated by *BWAFSQLi* with those of the original payloads, and further conducted a comparative analysis with three state-of-the-art mutation methods (WAM [13], DQNSQLi [38], and AdvSQLi [25]), thereby validating the effectiveness of *BWAFSQLi* in preserving semantic equivalence.
- RQ2 in Section 4.3.3: What is the impact of different mutation strategy selection methods on the performance of *BWAFSQLi*?
  The mutation strategy selection mechanism in *BWAFSQLi* consists of three core components: a random selection mechanism, a coordination mechanism driven by a *decay factor* and *historical data table*, and a *multi-position* simultaneous mutation mechanism. To evaluate the contribution of each component to the overall performance, we conducted a series of ablation experiments.
- RQ3 in Section 4.3.4: How effective is *BWAFSQLi* in attacking gray-box WAFs based on deep learning?
  To assess the effectiveness of *BWAFSQLi* in bypassing gray-box WAF based on deep learning, we conducted attack experiments on three representative models: CNN [17], LSTM [9], and GRU [22]. We then compared the performance of *BWAFSQLi* with three existing WAF bypassing methods, namely Priority Queue [13], Reinforcement Learning [38], and MCTS [25], in terms of bypass efficiency and bypass capability.
- RQ4 in Section 4.3.5: How effective is *BWAFSQLi* in attacking black-box WAFs?
  To further evaluate the effectiveness of *BWAFSQLi* in real-world scenarios, we conducted attack experiments on eight open-source or commercial black-box WAFs: ModSecurity, Chaitin Safeline, Huawei Cloud, Tianyi Cloud, Cloudflare, AWS, F5 and Fortinet. We then compared the results with two state-of-the-art WAF bypassing methods, Reinforcement Learning [38] and MCTS [25], to assess their respective bypass efficiency and bypass capability.
- RQ5 in Section 4.3.6: To what extent can *BWAFSQLi* effectively defend against malicious SQL injection payloads that successfully evade state-of-the-art WAFs?
  We documented the number and examples of mutated payloads that *BWAFSQLi* successfully bypassed across the eight categories of black-box WAFs described in RQ4. Furthermore, we proposed corresponding payload pre-processing defense strategies and conducted comparative experiments to evaluate the changes in FNR increase before and after applying these preprocessing defenses.
- RQ6 in Section 4.3.7: What is the distribution of different payload mutation strategies in *BWAFSQLi*?
  To analyze different payload mutation strategies to bypass a real practical WAF, we collected and evaluated combinations of mutation strategies that successfully bypassed the black-box WAF based on the process described in RQ4. Our analysis yielded several key findings.

4.1 Evaluation Metrics and Benchmarking Methods

4.1.1 Evaluation Metrics.

This section describes the attack testing process and the quantitative evaluation system for WAFs. In the initial attack phase, a total of N payload samples are sent to the target WAF. Among them, a payloads successfully bypass the WAF, while $b = N - a$ payloads are blocked. The blocked payloads are then subjected to mutation using the predefined *MPG* component. After each mutation generates a new payload, it is immediately tested

against the WAF again(each mutation corresponds to a single attack test). Let the total number of such follow-up tests be f, and the number of mutated payloads that successfully bypass the WAF be c, corresponding to r total HTTP requests.

To validate mutation correctness, we perform semantic equivalence verification between the original and mutated payloads using *WebSETest* (see Section 4.2). Both payloads are evaluated independently, and if the results match, the mutation is considered semantically equivalent. The number of such equivalent cases is recorded as e ($e \leq f$). An overview of the entire testing and mutation process is provided in Figure 12.

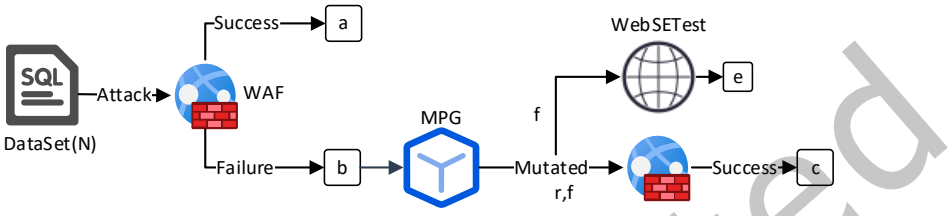

Fig. 12. Different number of attack payloads in the process of WAF injection bypass.

Based on this, the following evaluation indicators are defined as follows.

FNR_0 (Initial False Negative Rate(FNR) of the WAF): This metric evaluates the baseline detection capability of a WAF against initial attack payloads. It is defined as the proportion of payloads that successfully bypass the WAF during the initial attack phase relative to the total number of initial attack payloads. A lower FNR_0 indicates stronger initial detection capability of the WAF in identifying and blocking malicious payloads.

$$FNR_0 = \frac{a}{a+b} \tag{11}$$

MFNR (Mutation FNR of the WAF): This metric evaluates the resilience and robustness of a WAF against mutated attack payloads. It is defined as the proportion of payloads that successfully bypass the WAF during the mutation attack phase, combined with those that already bypassed it during the initial attack phase, relative to the total number of initial attack payloads. In the initial attack phase, a total of N payloads are tested against the WAF, among which a payloads successfully bypass and $b = N - a$ are blocked. During the subsequent mutation phase, each of the b blocked payloads is mutated and re-tested. If c of these mutated payloads succeed in bypassing the WAF, the total number of successful payloads becomes $a + c$. Accordingly, the number of still-blocked payloads is reduced from b to $b - c$. Thus, the MFNR metric can be expressed as the ratio of the total successful payloads $(a + c)$ to the total initial payloads $(a + b)$.

In our experiments, a bypass is considered successful only when the server returns status code 200; other status codes (e.g., 403, 500, 304) are not counted as successful bypasses. We define *false positives* as payloads that bypass the WAF but cause the target application to crash or return errors (e.g., HTTP 500), without achieving the intended exploit objective (such as data exfiltration or command execution). Such payloads are treated as *non-effective exploits* and excluded from the MFNR calculation, as they do not represent practically exploitable bypasses.

$$MFNR = \frac{a+c}{(a+c)+(b-c)} = \frac{a+c}{a+b} \tag{12}$$

IFNR (Incremental FNR of the WAF): This metric evaluates the bypass capability of the attack method and also reflects the WAF's adaptability to mutated payloads. It is defined as $IFNR = MFNR - FNR_0$, representing the difference between the false negatives during the mutation attack phase and the initial attack phase. A higher *IFNR* indicates that the mutation strategy employed by the attack method has a stronger ability to evade WAF

detection rules, and also implies that the WAF exhibits insufficient adaptability when confronted with complex and diverse mutated payloads.

$$IFNR = MFNR - FNR_0 = \frac{c}{a+b} \tag{13}$$

SPBARC (average number of requests required for one successful payload to bypass the WAF): It represents the average number of requests required for a single successful bypass. A lower *SPBARC* indicates lower request overhead and higher attack efficiency. Since network fluctuations, latency, etc., may cause inaccuracies in time-based metrics, and both computational time and wall-clock time scale approximately linearly with the number of requests. The total number of requests equals the number of payloads multiplied by the average requests per payload, thus the *SPBARC* metric stably reflects the attempts, computational effort, and time costs required for a bypass. If no payloads succeed, *SPBARC* is reported as ∞).

$$SPBARC = \frac{r}{c} \tag{14}$$

SER (Semantic Equivalence Rate): Measures the reliability of the mutation strategy in generating semantically equivalent payloads. It is defined as the ratio of the number of equivalent mutations to the total number of mutations. A *SER* value closer to *1* indicates a higher proportion of semantically equivalent payloads, reflecting greater effectiveness and robustness of the mutation process.

$$SER = \frac{e}{f} \tag{15}$$

Our primary objective is to quantify WAF robustness under adversarial payloads by maximizing the elevation of the *FNR* while minimizing the number of HTTP requests. We evaluate performance using two metrics: bypass efficiency, measured by *SPBARC*; and bypass capability, measured by *IFNR*.

4.1.2 Benchmarking Methods.

We selected three state-of-the-art methods as baseline approaches for comparison:

(1) WAF-A-MoLE (WAM): Demetrio et al. [13] defined eight mutation strategies based on regular expressions and guided the mutation process using a priority queue mechanism.
(2) DQNSQLi: Wang et al. [38] and Hemmati et al. [16] adopted the mutation strategies from [13] and proposed a reinforcement learning-based mutation selection method built on the gym-waf [30] framework.
(3) AdvSQLi: Qu et al. [25] introduced ten mutation strategies and represented SQLi payloads using a hierarchical tree structure. They employed a context-free grammar-based weighted mutation strategy to generate equivalent payloads and utilized MCTS to guide the mutation process.

4.2 To answer RQ1: Semantic equivalence assessment before and after SQL injection payload mutation

To conduct semantic equivalence verification, we regenerate 190 SQL injection payloads using a context-free grammar guided by a *convergence factor*, covering all attack types in Table 1. These payloads are then validated for equivalence on the custom platform *WebSETest*, built with HTML, MySQL, and PHP (see Figure 13).

WebSETest checks semantic equivalence by comparing the responses of SQL injection payloads before and after mutation. It submits both the original and mutated payloads to the PHP backend via an HTML form, where the *id* parameter is directly concatenated into an SQL query and executed using MySQL. We determine equivalence based on the response content—such as query results, error messages, numerical values (1/0), or blanks—following the rules in Table 7. If the responses match (for example, identical results, matching errors, equal numerical values, or consistent timestamps), the payloads are considered semantically equivalent.

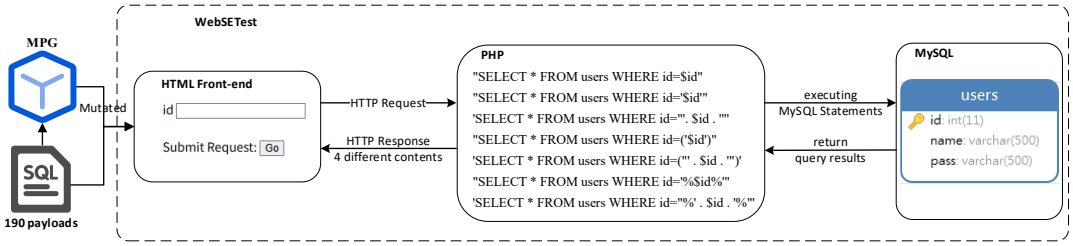

Fig. 13. The execution process of *WebSETest*.

Table 7. Response Content and Equivalence Judgment Rules.

Return content	Equivalence judgment basis
Query results	Query results are consistent before and after mutation → Semantic equivalence
SQL error message	Error messages are consistent before and after mutation → Semantic equivalence
Numerical value 1 (results found)/0 (no results found)	Numerical values are equal before and after mutation → Semantic equivalence
Blank content	Response times are inconsistent before and after mutation → Semantic non-equivalence (the blank content itself is the same, but the time difference reflects changes in execution)

As shown in Table 3, since DQNSQLi [38] and WAM [13] adopt the same payload mutation strategy, we selected one representative method—WAM [13]—along with the approach proposed by AdvSQLi [25] and our own *BWAFSQLi* for semantic equivalence comparison. The experimental results are presented in Table 8, where *Average SER* denotes the average semantic equivalence rate over 10 trials, and *Average Mutation Count* represents the mean number of payload mutations across those trials.

Table 8. Comparison of semantic equivalence experiment results.

	Average SER	Average Mutation Count
AdvSQLi	89.40%	1904
WAM	91.36%	1527
BWAFSQLi	100.00%	7856

As the results indicate, *BWAFSQLi* significantly outperforms both AdvSQLi [25] and WAM [13] in terms of mutation efficiency. It achieves a higher *Average Mutation Count* and a perfect semantic retention rate (*Average SER* = 100%), whereas the other two methods maintain an average semantic equivalence rate of approximately 90%.

This performance advantage is primarily attributed to two design choices in *BWAFSQLi*: (1) the optimized *Integer Encoding* strategy, which avoids applying hexadecimal encoding to numerical parameters of certain SQL functions (Table 4), ensuring correct execution; and (2) the application of URL encoding to GET requests during WAF attacks, which further preserves semantic equivalence.

4.3 Experiment Evaluation

4.3.1 Payload DataSet.

For the data requirements of RQ2 through RQ5, we employed two widely used benchmark datasets, as shown in Table 9: HPD [20] and SIK [31]. Specifically, HPD (HttpParamsDataset) [20] consolidates all SQL injection payloads from the CSIC dataset [32] and supplements them with injection samples generated using the sqlmap

tool [33]. Owing to its comprehensive coverage and high authenticity, HPD [20] has become a standard benchmark in SQL injection detection research [12, 18, 26]. The SIK [31], meanwhile, is sourced from the SIKgle platform and focuses on SQL injection attack data collected from public environments.

Table 9. Summary statistics of the dataset. The numbers before or after "/" indicate the number of benign/malicious payloads.

Datasets	Training	Validation	Testing	Total
HPD	11556/6508	3852/2169	3853/2170	30108
SIK	11613/6753	3871/2251	3872/2251	30611
SQLiCFG	6000/6000	2000/2000	2000/2000	20000

In addition, we constructed a custom dataset named *SQLiCFG*, consisting of attack payloads generated using a payload generation method based on context-free grammar driven by a *convergence factor*. *SQLiCFG* encompasses all attack types listed in Table 1 and includes a total of 20,000 samples (10,000 benign and 10,000 malicious payloads).

For all three datasets, we split the data in a 6:2:2 ratio, allocating 60% for training, 20% for testing, and 20% for validation.

4.3.2 *WAF Selection.*

We selected three mainstream gray-box WAFs based on deep learning, (CNN [17], LSTM [9], and GRU [22]), as well as eight open-source or commercial black-box WAFs, including the open-source ModSecurity and the commercial Changting Leichi WAF, Huawei Cloud WAF, Tianyi Cloud WAF, Cloudflare WAF, aws WAF, F5 WAF and Fortinet WAF, as attack test targets for *BWAFSQLi*. The specific information is shown in Table 10. Table 10 provides a comparison of various WAFs in terms of version, ruleset, result type, open-source status, and deployment method. Gray-box models typically produce score-based outputs, are open source, and are deployed locally. In contrast, black-box WAFs are mostly closed source, cloud-based, and produce binary outputs (0 or 1). In the Version row, the date following the backslash (e.g., /2025-06-16) indicates the most recent official update of the WAF at the time of evaluation, illustrating the recency of the version used in the experiment. A slash "/" in the table denotes that the item is either not applicable or the relevant information is unavailable.

Table 10. Information related to WAFs.

	Gray-box WAFs			Black-box WAFs(Practical WAFs)							
	CNN	LSTM	GRU	ModSecurity	Changting Leichi	Huawei Cloud	Tianyi Cloud	Cloudflare	AWS	F5	Fortinet
Verision	[24]	[26]	[27]	coreruleset /3.3.2	Community Edition /[8.9.0]-2025-6-5	Standard Edition /2025.5	Standard Edition /2025.4	Pro Edition /2025-06-16	/2025.10	/2025.10	/2025.10
Ruleset	/	/	/	1	High Intensity	Strict	Strict	All enabled and default	/	/	/
Result Type	Score			0, 1							
Open Source	Yes			No							
Deployment	Local			Cloud							

For gray-box WAFs, payloads are classified and assigned confidence scores, which are compared against a predefined threshold t. Scores above t are considered intercepted (i.e., not bypassed), while scores below t indicate successful bypasses. The threshold t thus plays a critical role in distinguishing between benign and malicious inputs. A lower t value reflects stricter detection criteria, making it more challenging for attack payloads to evade detection and increasing the difficulty of successful bypasses. To evaluate this, we adjusted the FNR and FPR metrics on three mainstream gray-box WAF models—CNN [17], LSTM [9], and GRU [22]—and determined their respective optimal threshold values t, with the results summarized in Table 11.

For black-box WAFs, we selected eight mainstream products for comparative evaluation to ensure representativeness across different deployment types and regions. Specifically, ModSecurity (open-source version, deployed on Nginx with CRS 3.3.2 at tolerance level 1) serves as a widely recognized baseline in both academia and

Table 11. The performance with different parameters in gray-box WAFs.

Gray-box WAF	Dataset	t	Accuracy(%)	Precision(%)	Recall(%)	FNR(%)	FPR(%)
CNN	SQLiCFG	0.004	99.5	99.5	99.5	0	1
		0.046	99.5	99.5	99.5	0	0.1
	HPD	0.03	99.34	99.35	99.34	0.05	1
		0.104	99.92	99.92	99.92	0.05	0.1
	SIK	0.015	99.1	99.1	99.1	0.71	1
		0.186	99.31	99.32	99.31	1.54	0.1
LSTM	SQLiCFG	0.081	99.425	99.43	99.425	0.15	1
		0.119	99.825	99.825	99.825	0.15	0.2
	HPD	0.046	99.32	99.33	99.32	0.05	1
		0.296	99.85	99.85	99.85	0.05	0.2
	SIK	0.085	98.81	98.81	98.81	1.51	1
		0.827	96.21	96.39	96.21	9.95	0.2
GRU	SQLiCFG	0.011	99.5	99.5	99.5	0	1
		0.137	99.9	99.9	99.9	0	0.2
	HPD	0.028	99.34	99.34	99.34	0.05	1
		0.162	99.85	99.85	99.85	0.05	0.2
	SIK	0.054	99.02	99.02	99.02	0.93	1
		0.291	99.36	99.36	99.36	1.38	0.2

industry. Changting Leichi WAF (community edition with enhanced rule sets) and Huawei Cloud WAF (standard edition with strict rules and deep anti-escape detection) represent mainstream commercial and open-source solutions in China, while Tianyi Cloud WAF (basic edition with strict rule sets) reflects the characteristics of large-scale state-owned cloud deployments. Cloudflare WAF (Pro version with all managed and OWASP rule sets) serves as a representative global cloud-based system, exemplifying international SaaS-based protection solutions. In addition, AWS WAF, F5 WAF, and Fortinet WAF respectively represent the world's most widely used cloud-native security service, enterprise-level load balancing and application protection product, and integrated network security solution. All three were tested under their default rule sets to cover three typical commercial protection systems: cloud-based, enterprise-grade hardware, and integrated security platforms. Overall, these systems encompass open-source, domestic commercial, and globally deployed enterprise-grade WAFs, forming a balanced and representative foundation for evaluation.

4.3.3 To answer RQ2: Ablation experiment analysis of different mutation strategy selection methods.
To thoroughly evaluate the effectiveness of different mutation strategy selection methods, we adopted the random selection approach proposed by Qu, Demetrio et al. [13, 25], referred to as *BWAFSQLi(R)*. In addition, we designed the following two methods for comparative experimentation:

(1) *BWAFSQLi(S)*: Initially adopts a random selection mechanism. In later stages, it is influenced by a coordination mechanism jointly driven by the *decay factor* and the *historical data table*, with strategy selection based on weight. However, it performs mutation at a single position only (if multiple positions match the same strategy, one is randomly selected for mutation).
(2) *BWAFSQLi*: Also begins with a random selection mechanism and later incorporates a coordination mechanism jointly driven by the *decay factor* and the *historical data table*. Additionally, it conducts a *multi-position* simultaneous mutation mechanism (if multiple positions match the same strategy, all of them are mutated).

We conducted comparative evaluations of the three methods on three deep learning–based gray-box WAFs and eight black-box WAFs deployed in real-world environments. In the experiments, using the *SQLiCFG* dataset, the parameters *Step* and *Budget* were set to 1 and 10, respectively. For gray-box WAFs, the minimum threshold t was used, while GET requests were employed for black-box WAFs. Detailed experimental results are presented in Tables 12. The *IFNR* of *BWAFSQLi(R)* is significantly lower than that of the other methods, while its *SPBARC* is higher than the other variants. This indicates that relying solely on random selection without any optimization of the strategy selection mechanism results in lower mutation efficiency. In contrast, *BWAFSQLi(S)*, which adopts

a random selection mechanism in the early stages and later incorporates a weight-based selection strategy driven by a *decay factor* and *historical data table*, achieves an improvement in *IFNR* and a notable reduction in *SPBARC*, thereby enhancing overall mutation efficiency. Furthermore, *BWAFSQLi* demonstrates the best performance among all methods: its *IFNR* is substantially higher than that of *BWAFSQLi(S)*, and its *SPBARC* is considerably lower. This shows that the introduction of the *multi-position* simultaneous mutation mechanism effectively reduces the number of required mutations, further improving mutation efficiency.

Table 12. *IFNR* and *SPBRC* in WAFs.

WAF	FNR_0	Metric	BWAFSQLi(R)	BWAFSQLi(S)	BWAFSQLi
CNN	0%	IFNR	0.06%	0.30%	1.28%
		SPBRC	8624	2495	309
LSTM	0.05%	IFNR	1.01%	2.31%	2.34%
		SPBRC	509	227	167
GRU	0.71%	IFNR	0.10%	0.27%	1.54%
		SPBRC	5178	2827	264
Modsecurity	0.12%	IFNR	0.26%	0.53%	3.78%
		SPBRC	1979	1411	53
Changting Leichi	2.67%	IFNR	0.44%	0.87%	1.23%
		SPBRC	1145	842	237
Huawei Cloud	5.13%	IFNR	1.25%	2.63%	2.94%
		SPBRC	387	197	118
Tianyi Cloud	2.84%	IFNR	1.17%	1.65%	4.67%
		SPBRC	426	282	55
Cloudflare	35.00%	IFNR	1.24%	2.23%	2.36%
		SPBRC	230	105	86
AWS	4.20%	IFNR	0.08%	0.07%	0.11%
		SPBRC	6101	6616	2789
F5	48.71%	IFNR	1.72%	1.72%	1.72%
		SPBRC	601	304	104
Fortinet	16.73%	IFNR	11.53%	13.50%	15.70%
		SPBRC	36	29	15

4.3.4 *To answer RQ3: Evaluation of BWAFSQLi on gray-box WAFs based on deep learning.*

To evaluate the effectiveness of *BWAFSQLi* on deep learning–based gray-box WAFs, we set different combinations of *Step* and *Budget* parameters, specifically 1-10, 1-20, and 10-10, to comprehensively examine the impact of different mutation rounds and frequencies on the *IFNR* and *SPBARC* performance metrics. The experiments used attack payloads from the *SQLiCFG*, HPD [20], and SIK [31] datasets, and, referring to the FPR and FNR metrics in Table 11, performed attack tests on three gray-box WAFs: CNN [17], LSTM [9], and GRU [22]. Furthermore, to validate the superiority of *BWAFSQLi*, we compared it with three state-of-the-art WAF bypass methods—WAM [13], DQNSQLi [16], and AdvSQLi [25]. The experimental results are presented in Tables 13 and 14.

As shown in Tables 13 and 14, the maximum bypass capability (*IFNR*) and single payload bypass cost (*SPBARC*) of *BWAFSQLi* exhibit a highly consistent trend across different gray-box WAFs. Figure 14 shows that when the *Step-Budget* increases from 1-10 to 1-20, the changes in both metrics are minimal—this is because *Budget=10* already covers the vast majority of combinations (strategy search space saturation) through 15 mutation strategies and a multi-position simultaneous mutation mechanism, and the *BWAFSQLi* method prohibits the reuse of selected strategies (strategy reuse constraint), leading to premature termination of the search. Further increasing the *Budget* does not improve performance; however, when the *Step-Budget* is adjusted from 1-10 to 10-10 (total *Budget = 10*, with the *Step* increasing from 1 to 10 for each *Budget* iteration), *IFNR* significantly improves (the increased *Step* density of mutation attempts within each *Budget* iteration) but *SPBARC* also rises simultaneously (the increased *Step* directly leads to a higher total number of requests), indicating that parameter optimization requires balancing bypass efficiency with request cost.

The experimental results indicate that *BWAFSQLi* significantly outperforms the three benchmark methods, namely WAM [13], DQNSQLi [38], and AdvSQLi [25], in terms of *IFNR*, particularly on the SIK dataset, where it achieves up to **93.39%**. Although AdvSQLi performs better than WAM and DQNSQLi in most cases, its *IFNR*

Table 13. *IFNR* comparison in gray-box WAFs.

Dataset	Gray-box WAF	FNR_0	Parameter: Step (1), Budget (10)				Parameter: Step (1),Budget (20)				Parameter: Step (10), Budget (10)			
			WAM	DQNSQLi	AdvSQLi	BWAFSQLi	WAM	DQNSQLi	AdvSQLi	BWAFSQLi	WAM	DQNSQLi	AdvSQLi	BWAFSQLi
SQLiCFG	$CNN^{1\%}$	0%	0.10%	0.00%	0.01%	**1.33%**	0.13%	0.00%	0.03%	**1.20%**	0.33%	0.00%	0.22%	**3.51%**
	$CNN^{0.1\%}$	0%	0.18%	0.00%	0.04%	**1.71%**	0.23%	0.01%	0.06%	**1.31%**	0.62%	0.01%	0.42%	**4.37%**
	$LSTM^{1\%}$	0.05%	0.89%	0.00%	0.13%	**1.79%**	1.07%	0.00%	0.12%	**1.85%**	0.66%	0.10%	1.55%	**2.48%**
	$LSTM^{0.2\%}$	0.05%	0.92%	0.00%	0.14%	**1.85%**	1.17%	0.00%	0.18%	**1.94%**	0.65%	0.00%	1.64%	**2.55%**
	$GRU^{1\%}$	0.71%	0.05%	0.00%	0.04%	**1.25%**	0.15%	0.00%	0.13%	**1.27%**	0.32%	0.00%	2.22%	**4.89%**
	$GRU^{0.2\%}$	1.54%	0.24%	0.02%	0.42%	**3.61%**	0.48%	0.05%	0.71%	**3.50%**	0.90%	0.02%	4.60%	**7.97%**
HPD	$CNN^{1\%}$	0.15%	0.09%	0.00%	0.01%	**0.26%**	0.20%	0.00%	0.12%	**0.22%**	0.80%	0.00%	1.37%	**2.68%**
	$CNN^{0.1\%}$	0.15%	0.30%	0.00%	0.08%	**0.45%**	0.32%	0.00%	0.24%	**0.46%**	1.19%	0.00%	1.85%	**3.59%**
	$LSTM^{1\%}$	0.05%	0.63%	0.00%	0.14%	**0.86%**	1.06%	0.00%	0.53%	**0.95%**	3.39%	0.00%	**7.87%**	3.13%
	$LSTM^{0.2\%}$	0.05%	0.90%	0.00%	0.36%	**1.12%**	1.48%	0.00%	0.80%	**1.69%**	4.50%	0.01%	**9.10%**	5.05%
	$GEU^{1\%}$	1.51%	0.23%	0.01%	0.09%	**0.53%**	0.46%	0.00%	0.27%	**0.53%**	1.55%	0.00%	**7.02%**	3.85%
	$GRU^{0.2\%}$	9.95%	0.39%	0.00%	0.28%	**1.86%**	0.65%	0.00%	0.82%	**1.96%**	2.68%	0.00%	**11.63%**	8.67%
SIK	$CNN^{1\%}$	0%	5.60%	0.02%	9.14%	**71.03%**	11.72%	0.02%	18.17%	**70.82%**	24.23%	0.50%	58.03%	**90.70%**
	$CNN^{0.1\%}$	0%	16.35%	0.68%	23.24%	**81.42%**	25.87%	0.68%	34.49%	**81.19%**	41.61%	0.63%	70.31%	**93.39%**
	$LSTM^{1\%}$	0.05%	5.75%	0.07%	19.80%	**77.64%**	8.85%	0.10%	36.78%	**77.84%**	18.84%	0.15%	79.19%	**88.72%**
	$LSTM^{0.2\%}$	0.05%	11.07%	0.05%	37.73%	**78.01%**	16.76%	0.01%	53.67%	**77.83%**	32.37%	0.83%	81.12%	**84.16%**
	$GRU^{1\%}$	0.93%	4.03%	0.24%	8.46%	**69.22%**	6.18%	0.03%	22.03%	**69.55%**	13.32%	1.26%	69.12%	**86.39%**
	$GRU^{0.2\%}$	1.38%	8.48%	0.06%	18.70%	**79.18%**	11.75%	0.45%	37.19%	**79.07%**	21.94%	1.08%	77.59%	**90.37%**

Table 14. *SPBARC* comparison in gray-box WAFs(If no payloads succeed, *SPBARC* is reported as ∞).

Dataset	Gray-box WAF	Parameter: Step (1), Budget (10)				Parameter: Step (1), Budget (20)				Parameter: Step (10), Budget (10)			
		WAM	DQNSQLi	AdvSQLi	BWAFSQLi	WAM	DQNSQLi	AdvSQLi	BWAFSQLi	WAM	DQNSQLi	AdvSQLi	BWAFSQLi
SQLiCFG	$CNN^{1\%}$	19992	∞	103928	**269**	30755	∞	39763	**290**	33414	∞	189692	**729**
	$CNN^{0.1\%}$	11099	∞	25962	**260**	17364	209901	19879	**296**	17748	307872	99195	**581**
	$LSTM^{1\%}$	2239	∞	7985	**219**	3720	∞	16233	**192**	16647	30659	26657	**773**
	$LSTM^{0.2\%}$	2165	∞	7413	**208**	3401	∞	10818	**209**	16911	∞	25168	**843**
	$GRU^{1\%}$	7998	∞	25980	**322**	7995	∞	15002	**317**	6615	∞	18482	**490**
	$GRU^{0.2\%}$	4759	54952	2473	**116**	5707	41950	2745	**118**	3801	153922	8738	**299**
HPD	$CNN^{1\%}$	21691	∞	119289	**1314**	19714	∞	17463	**1388**	6853	∞	38458	**1026**
	$CNN^{0.1\%}$	6774	∞	13253	**852**	12387	∞	8731	**805**	9342	∞	25224	**918**
	$LSTM^{1\%}$	3186	∞	7951	**475**	3763	∞	3913	**414**	3219	∞	5853	**1007**
	$LSTM^{0.2\%}$	2210	∞	5820	**346**	2702	∞	2609	**253**	2414	333847	5013	**620**
	$GRU^{1\%}$	8672	119254	11929	**735**	8666	∞	11947	**802**	7373	∞	6632	**811**
	$GRU^{0.2\%}$	5160	∞	3975	**236**	6191	∞	2550	**205**	4238	∞	3858	**344**
SIK	$CNN^{1\%}$	351	61679	119	**3**	330	117515	114	**3**	423	6090	464	**6**
	$CNN^{0.1\%}$	115	1573	46	**2**	140	4350	59	**2**	206	4717	265	**3**
	$LSTM^{1\%}$	337	15362	54	**2**	433	21284	55	**2**	556	20081	184	**5**
	$LSTM^{0.2\%}$	156	20002	25	**2**	202	226480	34	**2**	257	3348	88	**3**
	$GRU^{1\%}$	486	4546	128	**3**	630	78439	96	**3**	827	2356	293	**6**
	$GRU^{0.2\%}$	227	17550	57	**2**	323	4531	55	**2**	464	2740	193	**4**

Fig. 14. Average *IFNR* and average *SPBARC* trends of *BWAFSQLi* under different parameters.

still falls short of that achieved by *BWAFSQLi*. In terms of the *SPBARC*, *BWAFSQLi* also demonstrates a distinct advantage on the SIK dataset, achieving its best result with one successful bypass for every **two** HTTP requests. This reflects a substantially higher level of attack efficiency compared to the competing methods.

Notably, on the HPD dataset and under LSTM/GRU configurations with the *Step-Budget* set to 10-10, AdvSQLi temporarily surpasses *BWAFSQLi* in terms of *IFNR*. In addition, WAM's *IFNR* exceeds that of *BWAFSQLi* under the $LSTM^{1\%}$ configuration. These results are primarily due to the much higher number of mutation attempts used by AdvSQLi and WAM, with their *SPBARC* values being at least five times greater than that of *BWAFSQLi*.

To further evaluate the potential of *BWAFSQLi*, we conducted additional experiments with optimized parameters. Specifically, we increased the LSTM *Step* to 70 and the GRU *Step* to 30, while moderately raising the average request count (*SPBARC*) but keeping it well below the levels used by AdvSQLi and WAM. The results, shown in Table 15, demonstrate that as the average request count(*SPBARC*) increases, the *IFNR* of *BWAFSQLi* improves significantly and eventually surpasses both AdvSQLi and WAM. These findings further validate the superiority of *BWAFSQLi* in balancing bypass success and request efficiency.

Table 15. Performance comparison after parameter improvement in gray-box WAFs ("–" indicates that *BWAFSQLi* already outperforms WAM[13] in Table 13 and 14).

		SPBARC			**IFNR**			**Step-Budget**		
Gray-box WAF	Dataset	BWAFSQLi	AdvSQLi	WAM	BWAFSQLi	AdvSQLi	WAM	BWAFSQLi	AdvSQLi	WAM
$LSTM^{1\%}$	HPD	1007→2486	5853	3219	3.13%→7.98%	7.87%	3.39%	10-70	10-10	10-10
$LSTM^{0.2\%}$	HPD	620→1832	5013	-	5.05%→10.69%	9.10%	-	10-70	10-10	-
$GRU^{1\%}$	HPD	811→1130	6632	-	3.85%→7.82%	7.02%	-	10-30	10-10	-
$GRU^{0.2\%}$	HPD	344→591	3858	-	8.67%→14.28%	11.63%	-	10-30	10-10	-

4.3.5 *To answer RQ4: Evaluation of BWAFSQLi for black-box WAFs.*

To evaluate the effectiveness of *BWAFSQLi* in a black-box WAF scenario, we configured the *Step-Budget* parameter as 1-10. This configuration is designed to simulate realistic attack conditions while minimizing the number of requests required to achieve a successful bypass. The experiment utilized attack payloads from three datasets: *SQLiCFG*, HPD [20], and SIK [31]. These were tested against eight open-source or commercial black-box WAFs: ModSecurity, Changting Leichi WAF, Huawei Cloud WAF, Tianyi Cloud WAF, Cloudflare WAF, AWS WAF, F5 WAF, and Fortinet.

The evaluation were conducted using four request methods: GET, POST, GET(JSON), and POST(JSON). During the experiment, we observed that ModSecurity was entirely unable to detect malicious payloads in POST(JSON) requests, achieving (FNR_0) of 100% in initial tests. As a result, it was not possible to obtain mutable payloads under this configuration, and this combination was excluded from further evaluation. Furthermore, WAM [13] was excluded from the comparison because it lacks the capability to perform attacks in black-box WAF scenarios. Finally, we compared *BWAFSQLi* with two state-of-the-art methods: DQNSQLi [38] and AdvSQLi [25]. The experimental results are presented in Table 16, 17 and 18.

As shown in Table 16, 17 and 18, *BWAFSQLi* achieves significantly higher *IFNR* compared to the two benchmark methods, AdvSQLi and DQNSQLi, reaching up to 58.49% on the SIK dataset. In terms of *SPBARC*, *BWAFSQLi* also demonstrates the best performance on the SIK dataset, successfully bypassing the WAF with as few as two HTTP requests and thus greatly outperforming the other methods in attack efficiency.

Notably, in certain scenarios, AdvSQLi temporarily achieved a higher *IFNR* than *BWAFSQLi*: including POST and POST(JSON) requests on Huawei Cloud WAF with the HPD dataset, POST requests on F5 WAF with the SQLiCFG dataset, and GET requests on F5 WAF with the HPD dataset. This phenomenon is primarily attributable to AdvSQLi issuing substantially more request attempts, with its *SPBARC* nearly twice that of *BWAFSQLi*, thereby achieving IFNR over a larger number of requests.

Table 16. Evaluation of black-box WAFs effectiveness and bypass efficiency under the *SQLiCFG* dataset(If no payloads succeed, *SPBARC* is reported as ∞).

DataSet	WAF	Request Method	FNR_0	IFNR			SPBARC		
				BWAFSQLi	AdvSQLi	DQNSQLi	BWAFSQLi	AdvSQLi	DQNSQLi
SQLiCFG	ModSecurity	GET	0.12%	**3.99%**	0.93%	0.00%	**103**	1115	∞
		POST	0.09%	**3.93%**	0.92%	0.00%	**62**	1128	∞
		GET(JSON)	13.64%	**33.50%**	12.22%	0.00%	**3**	72	∞
		POST(JSON)	100.00%	0.00%	0.00%	0.00%	**0**	0	0
	Changting Leichi WAF	GET	2.67%	**1.11%**	0.64%	0.00%	**307**	1584	∞
		POST	2.01%	**2.20%**	0.74%	0.00%	**170**	1375	∞
		GET(JSON)	3.86%	**3.56%**	0.78%	0.01%	**64**	1290	138671
		POST(JSON)	2.01%	**1.09%**	0.52%	0.00%	**312**	1958	∞
	Huawei Cloud WAF	GET	5.13%	**3.09%**	0.10%	0.00%	**114**	9905	∞
		POST	5.18%	**13.46%**	0.97%	0.00%	**45**	1020	∞
		GET(JSON)	23.16%	**4.75%**	3.40%	0.07%	**18**	237	14786
		POST(JSON)	4.75%	**5.50%**	1.16%	0.02%	**47**	852	53843
	Tianyi Cloud WAF	GET	2.84%	**4.64%**	0.58%	0.00%	**60**	1728	∞
		POST	2.92%	**5.85%**	0.61%	0.00%	**57**	1651	∞
		GET(JSON)	6.64%	**4.82%**	0.83%	0.33%	**37**	1170	3256
		POST(JSON)	2.92%	**4.75%**	0.01%	0.24%	**57**	1831	4507
	Cloudflare WAF	GET	35.00%	**2.34%**	0.31%	0.02%	**78**	2157	51356
		POST	43.42%	**0.76%**	0.53%	0.02%	**77**	1144	51152
		GET(JSON)	25.57%	**3.01%**	0.24%	0.06%	**283**	3247	18229
		POST(JSON)	37.10%	**26.52%**	6.30%	0.08%	**4**	104	12758
	AWS WAF	GET	4.20%	**0.11%**	0.05%	0.00%	**2789**	16878	∞
		POST	2.69%	**0.38%**	0.30%	0.07%	**792**	2114	15512
		GET(JSON)	64.77%	**1.09%**	0.10%	0.00%	**106**	3756	∞
		POST(JSON)	63.18%	**1.46%**	1.05%	0.18%	**78**	367	5794
	F5 WAF	GET	48.71%	**1.76%**	0.68%	0.00%	**104**	768	∞
		POST	47.04%	3.59%	4.15%	0.20%	**46**	135	5071
		GET(JSON)	48.72%	**9.61%**	3.97%	2.04%	**16**	131	473
		POST(JSON)	47.47%	**8.04%**	7.91%	0.28%	**20**	71	3591
	Fortinet WAF	GET	16.73%	**15.70%**	11.35%	1.49%	**15**	74	678
		POST	15.97%	**15.78%**	12.41%	1.83%	**15**	69	541
		GET(JSON)	16.92%	**25.90%**	16.11%	0.06%	**9**	52	16347
		POST(JSON)	16.48%	**25.33%**	17.37%	0.49%	**9**	49	2098

To investigate this discrepancy, we conducted a comparative experiment by increasing the *Step* parameter of *BWAFSQLi* to 30 (see Table 19). The results indicate that when *BWAFSQLi*'s *SPBARC* is increased to a level comparable to AdvSQLi's, it not only achieves a significantly higher *IFNR*, but also maintains a lower request cost. This further confirms *BWAFSQLi*'s inherent advantage in improving attack efficiency through intelligent strategy search rather than relying on brute-force request accumulation.

4.3.6 To answer RQ5: How can BWAFSQLi defend against malicious SQL injection payloads?

To further investigate the real-world implications of the results presented in Section 4.3.5, we conducted a statistical analysis of the total number of payloads that successfully bypassed each black-box WAF. Specifically, we collected and tallied the number of payloads evaded by *BWAFSQLi*, yielding the following results: **12,287** for ModSecurity, **4,937** for Changting Leichi WAF, **7,768** for Huawei Cloud WAF, **6,193** for Tianyi Cloud WAF, **14,363** for Cloudflare WAF, **2,896** for AWS WAF, **12,448** for F5 WAF and **40,471** for Fortinet WAF. The detailed number of payloads for each request method and WAF is presented in Table 20.

Moreover, based on the seven preprocessing defense strategies described in Table 6, we conducted a comparative analysis of WAF performance before and after applying the preprocessing mechanisms, with the results shown in Table 21. Here, "Before" refers to the *IFNR* prior to preprocessing, "After" refers to the *IFNR* after preprocessing, and "Reduction" indicates the decrease in *IFNR* from Before to After. The analysis reveals that the preprocessing mechanisms substantially enhanced defense effectiveness, leading to a significant reduction in *IFNR*. Across all request types, the *IFNR* decreased by an average of **74.59%**. More specifically, when analyzed across different request types—including GET, POST, GET(JSON), and POST(JSON)—the *IFNR* exhibited an average reduction of at least **66.27%**, demonstrating a clear and consistent improvement in overall defensive capability.

Table 17. Evaluation of black-box WAFs effectiveness and bypass efficiency under the HPD dataset(If no payloads succeed, *SPBARC* is reported as ∞).

DataSet	WAF	Request Method	FNR_0	MFNR			SPBARC		
				BWAFSQLi	AdvSQLi	DQNSQLi	BWAFSQLi	AdvSQLi	DQNSQLi
HPD	ModSecurity	GET	0.42%	**2.75%**	1.59%	0.00%	**143**	690	∞
		POST	0.57%	**1.81%**	1.57%	0.00%	**151**	697	∞
		GET(JSON)	24.28%	**14.23%**	10.21%	0.32%	**4**	87	3213
		POST(JSON)	100.00%	0.00%	0.00%	0.00%	**0**	0	0
	Changting Leichi WAF	GET	7.57%	**4.12%**	2.38%	0.00%	**88**	451	∞
		POST	2.55%	**3.62%**	2.18%	0.00%	**70**	492	∞
		GET(JSON)	10.11%	**7.91%**	2.66%	0.04%	**25**	383	28974
		POST(JSON)	2.55%	**4.01%**	2.21%	0.00%	**68**	486	∞
	Huawei Cloud WAF	GET	0.92%	**1.09%**	0.92%	0.07%	**257**	1181	14839
		POST	0.57%	**2.32%**	**5.08%**	0.00%	**70**	214	∞
		GET(JSON)	12.83%	**19.64%**	6.27%	0.87%	**14**	152	1177
		POST(JSON)	0.57%	**2.18%**	**5.14%**	0.06%	**73**	212	16959
	Tianyi Cloud WAF	GET	5.60%	**3.94%**	0.90%	0.00%	**56**	1155	∞
		POST	5.32%	**3.91%**	0.73%	0.00%	**76**	1434	∞
		GET(JSON)	12.52%	**7.85%**	1.02%	0.56%	**27**	948	10492
		POST(JSON)	5.06%	**3.52%**	0.83%	0.00%	**75**	1261	∞
	Cloudflare WAF	GET	26.62%	**4.63%**	3.27%	0.01%	**53**	289	114806
		POST	26.25%	**5.83%**	2.89%	0.05%	**29**	272	22451
		GET(JSON)	23.77%	**4.57%**	2.81%	0.21%	**52**	357	4412
		POST(JSON)	23.55%	**35.57%**	13.82%	4.52%	**5**	60	231
	AWS WAF	GET	3.56%	**2.65%**	2.40%	0.00%	**119**	441	∞
		POST	2.77%	**3.21%**	3.00%	0.17%	**100**	356	6511
		GET(JSON)	54.92%	**2.57%**	0.69%	0.00%	**58**	715	∞
		POST(JSON)	48.33%	**6.72%**	2.39%	0.95%	**22**	215	1026
	F5 WAF	GET	36.77%	**2.27%**	**2.97%**	0.05%	**95**	233	22264
		POST	33.44%	**6.41%**	4.57%	0.38%	**32**	155	2837
		GET(JSON)	36.65%	**16.16%**	8.68%	0.00%	**11**	79	∞
		POST(JSON)	35.06%	**14.52%**	10.65%	1.29%	**12**	67	820
	Fortinet WAF	GET	8.57%	**16.82%**	16.07%	0.00%	**16**	62	∞
		POST	8.06%	**16.89%**	16.01%	0.05%	**16**	62	23031
		GET(JSON)	8.67%	**31.23%**	24.03%	7.51%	**8**	41	131
		POST(JSON)	8.14%	**30.15%**	24.29%	0.55%	**8**	40	1979

However, in a few cases, the IFNR did not drop to zero. Analysis of these cases revealed that they often employed the *Tautology Substitution* strategy, which is inherently challenging to mitigate effectively due to its reliance on logically irrefutable statements. For example, attackers can transform the simple tautology *1=1* into visually complex but semantically equivalent forms like *'2@HI{'='2@HI{'*. When it processed by the application, this becomes *SELECT * FROM users WHERE username='' AND password='' OR '2@HI{'='2@HI{'*, where the WAF's pattern matching may fail to detect the attack because the special characters (@, {, }) create unusual token patterns that evade signature detection, and although the strings appear distinct, database engines implicitly coerce them to identical numeric values during evaluation, rendering the expression logically true, while aggressive WAF filtering that normalizes input by removing special characters might further transform this into *2HI=2HI*—still a valid tautology that maintains the attack's effectiveness—demonstrating how such payloads can simultaneously evade signature-based detection, exploit database type conversion behaviors, and withstand naive input normalization attempts. Therefore, WAF vendors should pay particular attention to such payloads and promptly update their defense strategies to ensure more robust protection.

4.3.7 *To answer RQ6: The distribution of different payload mutation strategies used in assessing WAFs.*

To analyze the distribution of different payload mutation strategies in *BWAFSQLi*, we configured the *Step-Budget* parameter to 1-10 and conducted tests on eight black-box protection systems: ModSecurity, Changting Leichi WAF, Huawei Cloud WAF, Tianyi Cloud WAF, Cloudflare WAF, AWS WAF, F5 WAF and Fortinet WAF. These tests used attack payloads derived from the *SQLiCFG* dataset. The experiment quantified the distribution of mutation strategy combinations that successfully bypassed each WAF, thereby revealing the practical effectiveness of each individual strategy. The results is shown in Table 22, where N represents NON-JSON format and $\mathcal{J}$ represents JSON format. The symbol ● indicates that the strategy alone is sufficient to achieve a successful bypass. The

Table 18. Evaluation of black-box WAFs effectiveness and bypass efficiency under the SIK dataset(If no payloads succeed, *SPBARC* is reported as ∞).

DataSet	WAF	Request Method	FNR_0	MFNR			SPBARC		
				BWAFSQLi	AdvSQLi	DQNSQLi	BWAFSQLi	AdvSQLi	DQNSQLi
SIK	ModSecurity	GET	5.48%	**4.10%**	2.19%	0.15%	**86**	474	7144
		POST	5.72%	**2.72%**	1.49%	0.00%	**118**	693	∞
		GET(JSON)	31.15%	**37.76%**	12.60%	0.04%	**2**	59	28838
		POST(JSON)	100.00%	0.00%	0.00%	0.00%	**0**	0	0
	Changting Leichi WAF	GET	15.02%	**3.61%**	0.76%	0.00%	**56**	1201	∞
		POST	10.18%	**3.55%**	0.68%	0.00%	**67**	1365	∞
		GET(JSON)	18.04%	**7.00%**	0.89%	0.00%	**25**	1045	∞
		POST(JSON)	10.18%	**3.68%**	0.77%	0.00%	**59**	1192	∞
	Huawei Cloud WAF	GET	7.46%	**2.30%**	0.35%	0.18%	**143**	2959	6021
		POST	7.56%	**5.83%**	5.27%	0.57%	**24**	193	1865
		GET(JSON)	23.50%	**7.30%**	3.20%	1.05%	**34**	275	863
		POST(JSON)	7.51%	**5.46%**	5.29%	0.00%	**26**	193	∞
	Tianyi Cloud WAF	GET	6.74%	**3.39%**	0.44%	0.00%	**67**	2353	∞
		POST	6.77%	**4.15%**	0.43%	0.00%	**63**	2404	∞
		GET(JSON)	14.20%	**7.04%**	0.63%	1.52%	**34**	1498	3947
		POST(JSON)	6.74%	**4.10%**	0.48%	0.01%	**65**	2137	121236
	Cloudflare WAF	GET	24.81%	**10.03%**	1.53%	0.00%	**23**	524	∞
		POST	51.43%	**11.35%**	2.91%	0.01%	**10**	185	114881
		GET(JSON)	16.55%	**8.77%**	0.90%	0.00%	**34**	1021	∞
		POST(JSON)	42.67%	**50.05%**	15.41%	0.06%	**2**	40	107615
	AWS WAF	GET	6.36%	**1.85%**	0.90%	0.34%	**164**	1146	3153
		POST	17.52%	**2.13%**	2.04%	0.19%	**118**	449	5465
		GET(JSON)	62.28%	**2.78%**	0.37%	0.00%	**45**	1113	∞
		POST(JSON)	61.20%	**1.66%**	1.44%	0.19%	**74**	295	5201
	F5 WAF	GET	42.83%	**3.99%**	3.90%	0.00%	**47**	160	∞
		POST	41.82%	**5.28%**	4.73%	0.04%	**35**	134	22760
		GET(JSON)	40.87%	**24.58%**	11.21%	0.00%	**6**	57	∞
		POST(JSON)	40.37%	**18.42%**	11.91%	0.12%	**8**	54	8090
	Fortinet WAF	GET	19.24%	**40.38%**	19.96%	0.00%	**4**	43	∞
		POST	18.97%	**39.74%**	19.86%	1.31%	**4**	44	759
		GET(JSON)	20.31%	**58.49%**	24.02%	5.09%	**2**	35	143
		POST(JSON)	19.17%	**55.85%**	23.82%	1.39%	**2**	36	717

Table 19. Performance comparison after parameter improvement in black-box WAFs

			SPBARC		IFNR		Step-Budget	
Black-box WAF	**DataSet**	**Request Method**	BWAFSQLi	AdvSQLi	BWAFSQLi	AdvSQLi	BWAFSQLi	AdvSQLi
Huawei Cloud WAF	HPD	POST	70 → 179	214	2.32% → 8.88%	5.08%	10-30	1-10
Huawei Cloud WAF	HPD	POST(JSON)	73 → 156	212	2.18% → 8.49%	5.14%	10-30	1-10
F5 WAF	SQLiCFG	POST	46 → 101	5071	3.59% → 11.38%	4.15%	10-10	1-10
F5 WAF	HPD	GET	95 → 230	233	2.27% →7.12%	2.97%	10-10	1-10

Table 20. Distribution of *BWAFSQLi* Successful Bypasses across Different Request Methods and WAFs.

WAF	GET	POST	GET(JSON)	POST(JSON)	Total
Modsecurity	1158	895	10234	0	12287
Changting Leichi WAF	964	1013	2002	958	4937
Huawei Cloud WAF	686	1383	4298	1401	7768
Tianyi Cloud WAF	1272	1373	2229	1319	6193
Cloudflare WAF	1865	2311	1520	8667	14363
AWS WAF	506	626	700	1064	2896
F5 WAF	867	1648	5481	4452	12448
Fortinet WAF	7939	7883	12560	12089	40471
Total	15257	17132	39024	29950	101363

symbol ◐ denotes that the strategy must be combined with others to succeed, meaning it appears as the final step in the effective mutation sequence. A dash (–) signifies that the WAF does not detect the request method, while a blank space indicates that the strategy is ineffective against the corresponding WAF.

Based on the statistics in Table 22, we reclassify the proposed mutation strategy system into three effectiveness categories: *Highly effective* (Sum ≥ 20), *Ineffective* (Sum = 0), and *Moderately effective* (other cases). Specifically:

Table 21. *IFNR* of black-box WAFs before and after applying the preprocessing defense strategy. "–" indicates that the WAF did not perform detection for that request method.

DataSet	WAF	GET			POST			GET(JSON)			POST(JSON)		
		Before	After	Reduction	Before	After	Reduction	Before	After	Reduction	Before	After	Reduction
SQLiCFG	Modsecurity	3.99%	0.00%	100.00%	3.93%	0.04%	98.98%	33.50%	0.14%	99.58%	0.00%	0.00%	-
	Changting Leichi WAF	1.11%	0.05%	95.50%	2.20%	0.11%	95.00%	3.56%	0.08%	97.75%	1.09%	0.14%	87.16%
	Huawei Cloud WAF	3.09%	0.18%	94.17%	13.46%	0.11%	99.18%	4.75%	0.35%	92.63%	5.50%	0.44%	92.00%
	China Telecom Cloud WAF	4.64%	0.19%	95.91%	5.85%	0.21%	96.41%	4.82%	0.43%	91.08%	4.75%	0.39%	91.79%
	Cloudflare WAF	2.34%	1.07%	54.27%	0.76%	0.14%	81.58%	3.01%	0.37%	87.71%	26.52%	0.26%	99.02%
	AWS WAF	0.11%	0.11%	0.00%	0.38%	0.32%	15.79%	1.09%	0.34%	68.81%	1.46%	1.25%	14.38%
	F5 WAF	1.76%	0.04%	97.73%	3.59%	1.63%	54.60%	9.61%	0.87%	90.95%	8.04%	1.24%	84.58%
	Fortinet WAF	15.70%	0.64%	95.92%	15.78%	1.02%	93.54%	25.90%	1.57%	93.94%	25.33%	0.69%	97.28%
HPD	Modsecurity	2.75%	0.97%	64.73%	1.80%	0.81%	55.00%	24.28%	0.35%	98.55%	0.00%	0.00%	-
	Changting Leichi WAF	4.12%	0.20%	95.15%	3.62%	0.37%	89.78%	7.91%	0.37%	95.32%	4.01%	0.37%	90.77%
	Huawei Cloud WAF	1.09%	0.92%	15.63%	2.32%	0.77%	66.72%	19.64%	0.30%	98.50%	2.18%	0.79%	63.62%
	China Telecom Cloud WAF	3.94%	0.03%	99.32%	3.91%	0.29%	92.61%	7.85%	0.18%	97.65%	3.52%	0.08%	97.68%
	Cloudflare WAF	4.63%	2.64%	42.98%	5.83%	3.19%	45.28%	4.57%	2.69%	41.14%	35.57%	1.09%	96.94%
	AWS WAF	2.65%	2.43%	8.30%	3.21%	2.50%	22.12%	2.57%	0.94%	63.42%	6.72%	5.34%	20.54%
	F5 WAF	2.27%	0.10%	95.59%	6.41%	3.19%	50.23%	16.16%	1.19%	92.64%	14.52%	1.64%	88.71%
	Fortinet WAF	16.82%	0.33%	98.04%	16.89%	0.61%	96.39%	31.23%	1.80%	94.24%	30.15%	0.49%	98.37%
SIK	Modsecurity	4.10%	1.01%	75.37%	2.72%	1.03%	62.13%	37.76%	1.10%	97.09%	0.00%	0.00%	-
	Changting Leichi WAF	3.61%	0.17%	95.29%	3.55%	0.45%	87.32%	7.00%	0.34%	95.14%	3.68%	0.61%	83.42%
	Huawei Cloud WAF	2.30%	1.35%	41.30%	5.83%	1.09%	81.30%	7.30%	0.08%	98.90%	5.46%	0.67%	87.73%
	China Telecom Cloud WAF	3.39%	0.64%	81.12%	4.15%	1.36%	67.23%	7.04%	0.76%	89.20%	4.10%	1.38%	66.34%
	Cloudflare WAF	10.03%	5.85%	41.67%	11.35%	3.26%	71.28%	8.77%	4.29%	51.08%	50.05%	3.01%	93.99%
	AWS WAF	1.85%	1.58%	14.59%	2.13%	1.86%	12.68%	2.78%	1.10%	60.43%	1.66%	1.54%	7.23%
	F5 WAF	3.99%	0.58%	85.46%	5.28%	0.55%	89.58%	24.58%	4.64%	81.12%	18.42%	1.16%	93.70%
	Fortinet WAF	40.38%	2.73%	93.24%	39.74%	3.03%	92.38%	58.49%	4.10%	92.99%	55.85%	3.41%	93.89%

Table 22. Distribution of the effectiveness of mutation strategies that bypass WAFs.

WAF	Request Method	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10	M11	M12	M13	M14	M15
ModSecurity	GET					◐	◐			●						
	GET(JSON)		●							●						
	POST		◐			◐				●						
	POST(JSON)	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
Changting Leichi	GET					●			◐	●			◐		●	◐
	GET(JSON)						●			◐	●		●	●	●	●
	POST					●	●	◐		●		●		◐	●	◐
	POST(JSON)				●	●			◐	●				◐	●	◐
Huawei Cloud	GET		◐			●	●								◐	
	GET(JSON)		●				●		◐		●	●		●	●	●
	POST		●			●	◐			●		◐			●	
	POST(JSON)		◐		●	●	●		◐	●		◐	◐		●	
Tianyi Cloud	GET		●		◐		●					●	●		●	◐
	GET(JSON)						●			◐	●	●	●	●	●	
	POST		◐				●			◐			●		●	◐
	POST(JSON)						●			●		●	●		●	
Cloudflare WAF	GET				◐	◐	◐		●			●		●	●	●
	GET(JSON)				◐		◐							●	●	●
	POST		◐			●	●	●		●		●		●	●	●
	POST(JSON)	●	●		◐	◐	◐	◐	◐	◐	◐	●	●	◐	◐	◐
AWS	GET									●				●	●	
	GET(JSON)						●	●		●					●	
	POST				◐	◐				◐		●		◐		●
	POST(JSON)	●	●			●	●			●		●	●	●	●	●
F5	GET					●	◐			◐			●			
	GET(JSON)		●			●	◐	●		●	●		●	●	◐	
	POST	●	●		●	●	●					●	●		◐	●
	POST(JSON)	◐	●		◐	●	●	◐		●		◐	●	●	◐	
Fortinet	GET		●		◐	◐	◐	◐	●	●		●	●	◐	◐	◐
	GET(JSON)	◐	●		◐	◐				●		●	●	◐	●	
	POST	◐	●		◐	●	●	●	●	●	◐	●	●	●	●	◐
	POST(JSON)	●	●			●	●	●	◐	●	◐	●	●		●	
Effectiveness Division	●	4	13	0	3	14	16	5	3	19	4	15	15	11	20	8
	◐	3	5	0	9	7	7	4	6	6	3	3	2	11	6	8
	Sum	7	18	0	12	21	23	9	9	25	7	18	17	22	26	16

- *Highly effective* (Sum $\geq$ 20): Includes Integer Encoding (Sum = 21), Equal Swapping (Sum = 23), Comment Injection in Whitespace (Sum = 25), Where Rewriting (Sum = 22), and Tatology Substitution (Sum = 26). These strategies demonstrate consistently strong and robust bypass capabilities across different WAFs, indicating their high standalone and combined effectiveness.
- *Moderately effective*: Includes Keyword Case Swapping (Sum = 7), Whitespace to Control Character (Sum = 18), Comment Extension (Sum = 12), And Logical Invariant (Sum = 9), Or Logical Invariant (Sum = 9), And Substitution (Sum = 7), Or Substitution (Sum = 18), Keyword Inline Comment (Sum = 17), and QuotationMark Encoding (Sum = 16). These strategies achieve partial success either independently or when combined with other strategies.
- *Ineffective* (Sum = 0): Comment Rewriting (Sum = 0). This strategy failed to produce any successful bypasses, either alone or in combination, suggesting negligible impact on evading WAF detection.

Different WAFs show large differences in how well they defend against evasion, mainly because their signature and pattern detectors differ. Both highly effective and moderately effective mutation strategies can still bypass many WAFs. For example, *Tautology Substitution* and *Comment Injection in Whitespace* create payloads that are semantically equivalent but structurally diverse, which greatly raises the chance of successful evasion. *Whitespace to Control Character*, *Equal Swapping*, and *Integer Encoding* combine well with other strategies and strengthen obfuscation. Although these strategies depend on specific syntactic features, their varied transformations let them keep relatively stable bypass performance across different WAF types.

Our two new strategies, *Quotation Mark Encoding* and *Comment Extension*, show promising results and classify as moderately effective. In single-use tests, *Comment Extension* bypassed two request types and *Quotation Mark Encoding* bypassed five. In combined-use tests, *Comment Extension* rose to four and *Quotation Mark Encoding* to six bypassed request types. *Quotation Mark Encoding* breaks string boundaries by changing how quotes are encoded or represented, which defeats WAFs that rely on static patterns or simple lexical checks. *Comment Extension* hides attacks by extending or moving comments to break statement structure. Neither strategy rates as highly effective, but both are robust and useful alone or in combination.

By contrast, *Comment Rewriting* (Sum = 0) proved ineffective, as it only alters the style of comments without significantly affecting syntax or semantics. In our experiments, it never achieved single-step evasion and never appeared as the final step in any successful mutation chain, offering little practical value for bypassing WAFs. Other strategies, such as *Keyword Case Swapping*, show moderate effectiveness: while their overall impact is limited (low Sum value), they can still be helpful in certain cases. It should be noted that modern WAFs typically perform case-insensitive matching, so *Keyword Case Swapping* alone rarely improves evasion success. Similarly, *And Substitution* demonstrates moderate effectiveness: by replacing logical operators with alternative forms (e.g., using && instead of AND), it introduces slight syntactic variations that may aid evasion in specific situations, but its overall impact is limited.

5 Related Work

We compare BWAFSOLi with existing methods (WAM [13], DQNSQLi [16, 38], and AdvSQLi [25]) across three aspects: dataset limitations, mutation-based search methods, and semantic equivalence. Table 23 shows the results: Scenario lists evaluable WAF types (dataset applicability), Datasets specifies used datasets (dataset limitations), Search Strategy/Mutate Operations/Defense detail search methods (technique and defense differences), and Equivalence examines handling of semantic-preserving adversarial examples.

Table 23. Comparative Analysis with Prior Work.

	Scenario	DataSet	Mutate Operations	Search Strategy	Defense	Equivalence
WAM [13]	Gray-box	wafamole-dataset(randgen [19]), sqlmap and OWASP ZAP	10	Priority Queue	No	No
AdvSQLi [25]	Black-box, Gray-box	HPD [20], SIK [31]	13	MCTS	No	No
DQNSQLi [16, 38]	Black-box, Gray-box	Not specified	10	Reinforcement Learning	No	No
BWAFSQLi	Black-box, Gray-box	SQLiCFG(CFG rules), HPD [20](CSIC dataset and sqlmap), SIK [31](SIKgle platform)	15	Decay factor, History Data Table, Multi-position Simultaneous Mutation	Yes	Yes

5.1 DataSet Limitations

In web attack research, SQL injection and related attacks depend on carefully crafted payloads to exploit vulnerabilities. However, existing payloads are predominantly sourced from public datasets or generated manually and through automated tools—both approaches exhibit significant limitations.

5.1.1 Public Datasets.

Demetrio et al. [13] used SQLi samples from Sqlmap and OWASP ZAP, however, these samples show limited structural and semantic diversity, making it difficult to generalize well across different WAF environments. Qu et al. [25] uses typical payload corpora such as HPD [20] and SIK [31] have obvious flaws: HPD [20] contains mislabeled samples (misclassifying benign strings like "5739-5839" and "1wwis" as malicious in Fig. 3), while SIK [31] uses full SQL statements as payloads, and the hardcoding of keywords introduces many redundant features, resulting in poor effectiveness of SQL injection–based adversarial attacks for evaluating WAFs.

To address these issues, *BWAFSQLi* implements two practical design improvements. First, based on a context-free grammar, *BWAFSQLi* systematically extract and consolidate syntactic fragments from real attack logs, tool-generated samples (e.g., Sqlmap, OWASP ZAP), and public datasets (HPD [20] and SIK [31]), Thereby significantly expanding syntactic and semantic coverage, covering a broader range than existing methods [13, 16, 25, 38]. Second, *BWAFSQLi* focuses on the variable parts of SQL statements rather than blindly mutating entire queries, producing more representative and targeted adversarial payloads and substantially improving bypass performance in both black-box and gray-box settings.

5.1.2 Payload Generation.

Existing context-free grammar methods (e.g., Appelt et al. [5, 6]) generate common payloads (e.g., integer encodings, quote variants, UNION queries, blind injections) but omit key categories (parenthesized expressions, percent-encoding, tautologies, stack/error-based injections). Demetrio et al. [13] generate executable queries via randgen [19] with real table/column names, yet their standalone queries lack URL/form/JSON/HTTP context, failing to simulate context-sensitive obfuscation or realistic exploitable scenarios.

In comparison, *BWAFSQLi* builds upon the work of Appelt et al. [5, 6] and Demetrio et al. [13], employing a context-free grammar to expand the syntactic and semantic coverage of high-quality, diverse payloads. During payload generation, *BWAFSQLi* utilizes more comprehensive generation rules to construct payloads that can be properly closed and concatenated within real URL, form, or JSON HTTP contexts to trigger backend logic, thereby significantly enhancing payload executability and practical effectiveness.

5.2 Mutation-Based Search Approaches

Payload tokens typically map to multiple mutation operations, each yielding diverse string substitutes. Exhaustively exploring all combinations incurs high computational and HTTP request costs, necessitating efficient search strategies to pinpoint viable WAF-bypassing mutations in this vast space. Current research primarily

focuses on Low-maliciousness-score or Random based Search strategies and reinforcement learning based Search strategies.

5.2.1 Low-maliciousness-score or Random-based Search strategies.

Studies [13, 25] show mutating low-maliciousness-score payloads reduces detection risks. Demetrio et al.'s WAF-A-MoLE [13] uses a priority-queue-based pipeline: it iteratively mutates initial payloads to generate candidates, then selects the next seed by ascending maliciousness score until bypassing the WAF or meeting termination conditions. However, non-linear score behavior—where low-score payloads may increase in score post-mutation—reduces bypass efficiency. In addition, this method relies on the score feedback available in gray-box settings and cannot be directly applied to black-box scenarios. Qu et al. [25] in AdvSQLi model SQLi payloads as a hierarchical syntax tree (nodes represent mutable fragments like literals, identifiers, or subexpressions) and apply Monte Carlo Tree Search (MCTS) to explore mutation paths: selecting nodes to expand (biased to low scores or random), generating child nodes via random mutations, producing full candidate payloads through randomized rollouts for evaluation and scoring, then backpropagating scores to update tree statistics and guide subsequent selection/expansion. However, this method incurs high computational/request overhead for deep/highly-branched paths, limiting its use in real-time or black-box scenarios. Moreover, refs [13, 25] commonly rely on random selection of mutation operations during payload mutation, which can unintentionally increase payload maliciousness—thus raising detection risk, generating more ineffective attempts, and increasing the computational and request cost of bypassing.

Thus, *BWAFSQLi* dynamically adjusts strategy weights using historical bypassing success rates and employs a decay factor together with a history data table driven multi-location mutation mechanism. By prioritizing mutation strategies that have performed well, it avoids inefficient search paths and accelerates convergence, producing more efficient and reliable bypasses in both black-box and gray-box evaluations.

5.2.2 Reinforcement Learning based Search strategies.

Compared to low-maliciousness-score or random search strategies, researchers modeled attack payload generation as a sequential decision problem and applied RL to optimize mutation strategy selection, addressing combinatorial explosions and search-efficiency bottlenecks caused by SQL injection complexity and mutation diversity [16, 38]. Typical approaches build a state space (encoding payload features), design an action space (available mutation strategies), and define a reward function (based on bypass outcomes), enabling agents to approximate optimal policies via interactive training. Wang et al. [38] adapted gym-malware's approach, using three-level histogram vectors for payload states, mapping [13]'s strategies to actions, and training with PPO. Hemmati et al. [16] expanded the strategy set, added FastText embeddings for better state representation, and enhanced training with prioritized experience replay, random network distillation, double Q-learning, and PPO. However, despite significantly improving bypass rates, RL incurs substantial costs: complex state-action-reward design, reliance on large labeled/interaction data and compute resources, and excessive HTTP requests during training. Additionally, learned policies are often hard to interpret, reducing controllability and reproducibility.

In contrast, *BWAFSQLi* eliminates the costly explicit training pipeline. It parses the structure of attack payloads to build a mutation library and employs a decay-driven mechanism to dynamically adjust strategy weights, allowing it to prioritize mutation operations with higher historical success rates to achieve fast convergence and RL-like adaptive behavior. Compared to pure RL approaches, it significantly reduces computational and request overhead, offers transparent and interpretable strategies, and maintains robust bypass performance in both black-box and gray-box scenarios.

5.3 Semantic Equivalence Issues

Payload mutations are typically achieved by adding, modifying, or removing malicious fragments that are easy to detect, but such changes can break the payload's attack semantics or syntactic integrity, causing the payload to fail. In fact, this risk to semantic equivalence can arise both from the inherent limitations of simple string replacement methods and from specific mutation operations themselves.

5.3.1 Limitations of simple string replacement.

Common string-replacement methods used in prior work [13, 16, 38] primarily rely on regular expressions for substitutions, which have several limitations. According to the Chomsky hierarchy [10], the class of languages describable by regular expressions corresponds to finite automata and cannot fully capture the context-free or context-sensitive structures commonly found in programming languages or attack payloads (for example, SQL injection payloads). As a result, purely string-based mutations may miss mutable parts that are crucial to payload semantics, or even unintentionally break the original functionality of an SQL injection payload by, for instance, destroying matching quotes. Given these intrinsic limitations, there is an urgent need to introduce syntax- and semantics-aware mutation strategies that increase variant coverage while preserving executability. To this end, *BWAFSQLi* adopts semantics-aware string replacement: it first decomposes a payload into syntactic tokens (for example string literals, identifiers, numeric literals, quote markers, comment fragments, operators, and encoded segments), then maps one or more applicable mutation strategies to each token type as targets for subsequent token-level transformations. By constraining mutations to strategies that are appropriate for the corresponding token types and that preserve semantics, *BWAFSQLi* can generate functionally equivalent and executable variants, thereby improving coverage and providing high-fidelity inputs for more reliable WAF evaluation and attack generalization.

5.3.2 Non-equivalence issues in mutation operations.

Prior work [13, 16, 25, 38] often employs mutation techniques such as integer encoding to increase payload diversity. However, studies show that when such mutations are applied to critical function parameters that require strict types or numeric values (for example, sleep(), see Section 3.3 for details), semantic equivalence can be broken and the payload may fail. In addition, strategies proposed by Hemmati et al. [16]—such as keyword repetition (e.g., turning select into seselectlect) and keyword splitting (e.g., turning select into CONCAT('se','lect'))—expand token-level mutation coverage but, in many cases, severely distort the original keyword semantics and render payloads nonfunctional. To preserve functional integrity, *BWAFSQLi* detects and skips function parameters that are sensitive to type or numeric constraints before applying integer encoding. Qu et al. [25] also identified an issue where special characters in HTTP GET requests were not URL-encoded according to standards, which can cause parameter truncation or parsing errors and thus undermine experimental reliability. To address this, *BWAFSQLi* normalizes URL encoding before mutation to ensure semantic and executable equivalence pre- and post-mutation.

To further improve the executability and stability of mutated payloads, *BWAFSQLi* introduces two new mutation operations (see Table 3 for a detailed comparison with the mutation operations in [13, 16, 25, 38]): quote encoding (e.g., 'user' ⟶ 0x75736572) and comment extension (e.g., 1' or 1=1# ⟶ 1' or 1=1#hello). Both operations are designed to preserve semantic equivalence, thereby increasing diversity while ensuring payload executability and experimental reliability.

Moreover, *BWAFSQLi* proposes seven lightweight preprocessing defense strategies not covered in previous works [13, 16, 25, 38], further enhancing the detection and protection capabilities of WAF vendors.

6 Discussion

6.1 Ethics and Responsible Use

This study focuses on systematically evaluating the robustness of WAFs against adversarial SQL injection attacks. We recognize that adversarial sample generation and mutation techniques, if released without safeguards, could potentially be misused. To mitigate such risks, we adhere to the following ethical principles:

6.1.1 Responsible Disclosure.

Prior to publication, we have notified the relevant WAF vendors of the rule vulnerabilities and bypass strategies discovered in our research, allowing them sufficient time to patch or strengthen their products. We intentionally avoid disclosing complete exploit payloads or attack scripts that could be directly used for malicious purposes. We submitted vulnerability reports and contacted technical support personnel and security researchers from the affected vendors via email to share our findings.

6.1.2 Research-Only Purpose.

The proposed *BWAFSQLi* framework is intended solely to support academic and industrial research aimed at improving WAF defense capabilities. All experiments were conducted in controlled environments and test platforms to ensure that no real production systems or user data were affected.

6.1.3 Balancing Transparency and Security.

To promote academic exchange while ensuring safety, we maintain necessary transparency regarding our research methods while avoiding disclosure of attack details that could be directly exploited. We provide only abstract descriptions and methodological explanations; for the bypassable SQL injection payloads presented in this paper, all mutated parts are represented as "□" (Table 24) to minimize potential misuse. To ensure the transparency of *BWAFSQLi*, the self-built dataset *SQLiCFG* and the *WebSETest* platform used to verify the semantic equivalence of payload mutations are publicly available, providing foundational resources for related research.

6.1.4 Promoting Defensive Value.

In addition to generating adversarial attacks, we also propose pre-processing defense strategies (Table 6) to help WAFs resist adversarial mutations. We encourage WAF developers and security practitioners to adopt these defensive measures, ensuring that our research outcomes serve primarily constructive and protective purposes.

6.2 Case Study

To more clearly demonstrate the effectiveness of the payloads, we further analyze the vulnerabilities through concrete case studies. We selected five representative original payloads (shown in Table 24) from that can exfiltrate table names, column names, and other sensitive information. Unsurprisingly, if these original payloads are sent directly to the web service, Huawei Cloud WAF will block them. We then fed the above payloads into *BWAFSQLi* (assuming a web injection point has been identified and that the request is a GET request carrying JSON). Under Huawei Cloud WAF protection, the original payloads could not perform the injection; however, after mutation by *BWAFSQLi*, the generated mutated payloads (also shown in Table 24) successfully executed the injection, obtaining the target (sqli-labs [2]) database schema and portions of sensitive data (shown in Figure 15). This case demonstrates that *BWAFSQLi* can reveal WAF detection blind spots in real-world scenarios and provide actionable samples for improving detection rules.

[2]https://github.com/Audi-1/sqli-labs

Table 24. Several example payloads that can bypass Huawei Cloud WAF and pose significant harm. To minimize potential misuse, all mutated portions are masked by "□".

No.	Origin payloads	Mutated payloads (Mask)	Leaked information
1	-1' or 1=1−+	-1'□□or□1=1−+□	All records from the target table (if echoed)
2	-1' union select database(),version(),user()−+	-□'□□□□□database(),version(),user()−+	Database name, DBMS version, current database user
3	-1' union select 1,2,group_concat(table_name) from information_schema.tables where table_schema=database()−+	-□'□□□□□1,2,group_concat(□) □□□information_schema.tables □□□table_schema like database()−+□	List of table names in the current database
4	-1' union select 1,2,group_concat(column_name) from information_schema.columns where table_name='users' and table_schema=database()−+	-1'□union□select□1,2,group_concat(column_name)□ from□information_schema.columns □where□table_name='users'□and□table_schema=database()−+	List of column names in the users table
5	-1' union select group_concat(id), group_concat(name),group_concat(pass) from users−+	-1'□□□□□group_concat(id), group_concat(name),group_concat(pass)□□□users−+□	All id, name, and pass values from the users table

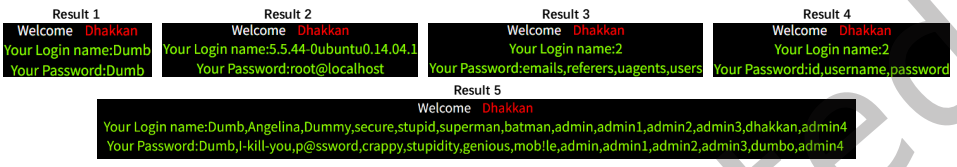

Fig. 15. Information leaked by the payload instance, arranged in the order of Table 24.

6.3 BWAFSQLi Advantages

The general and extensible WAF attack framework designed in this study, *BWAFSQLi*, demonstrates multidimensional core advantages. In terms of bypass effectiveness and compatibility, *BWAFSQLi* achieves efficient breakthroughs against deep learning–based gray-box WAFs as well as real-world open-source (e.g., ModSecurity) and commercial (e.g., Chaitin Leichi, Huawei Cloud, Tianyi Cloud, and Cloudflare) WAFs, increasing the WAF FNR by up to **93.39%**. Meanwhile, by implementing seven preprocessing defense strategies, experiments on eight real-world WAFs show that the average reduction in FNR growth reaches **74.59%**. More specifically, when analyzed across different request types—including GET, POST, GET(JSON), and POST(JSON)—the *IFNR* exhibited an average reduction of at least **66.27%**, validating *BWAFSQLi*'s broad adaptability and significant protective effectiveness across multiple platforms.

Innovative attack payload generation design is another standout feature of *BWAFSQLi*. *BWAFSQLi* proposes 58 grammar rules covering 18 types of SQL injection attacks, significantly expanding the limitations of traditional datasets in terms of attack pattern diversity. Additionally, it develops a mutation system composed of 26 grammar rules, encompassing 15 refined mutation strategies (such as *Quotation Mark Encoding* and *Comment Extension*). while ensuring that the mutation process strictly maintains semantic equivalence and the integrity of malicious functionality, it achieves a synergistic optimization of payload complexity and diversity. This design enables the payloads generated by *BWAFSQLi* to have a universal syntax structure, which facilitates structural division and feature extraction (enhancing the reliability of the dataset and the accuracy of annotations) while also supporting users in customizing attack types based on their needs and generating diverse mutation candidate values under semantic constraints, fully meeting personalized experimental requirements.

At the technical breakthrough level, *BWAFSQLi* proposes two new mutation strategies: *Quotation Mark Encoding* (converting quotation marks and their internal content into hexadecimal format—for example, *user* becomes *0x75736572*, effectively bypassing detection mechanisms based on quotation marks and keywords) and *Comment Extension* (appending redundant comments at the end of the payload—for instance, extending *1' or 1=1#* to *1' or 1=1#hello* to interfere with WAF rule matching logic), significantly improving bypass efficiency. At the same time, in response to the semantic integrity issues of traditional predefined string replacement methods, *BWAFSQLi* designs payload generation method based on context-free grammar guided by a *convergence factor*.

By dynamically adjusting generation rules, it enhances the structural complexity of mutated payloads while preserving their semantics. It also strictly limits the use of the integer encoding strategy, which may lead to functional degradation, and precisely handles special characters in URL encoding to ensure proper parsing during network transmission. In addition, *BWAFSQLi* includes a semantic equivalence verification web platform, *WebSETest*, built with HTML, PHP, and MySQL. This platform enables efficient verification processes, ensuring the semantic consistency of mutated payloads from a technical perspective.

In terms of mutation process optimization, *BWAFSQLi* proposes a mutation strategy selection method collaboratively driven by a *decay factor* and a *historical data table*. It dynamically balances exploration and exploitation by applying random selection during the early stage and weight-based precise selection in the later stage based on historical data. This design addresses the high computational complexity and frequent invalid attempts found in traditional approaches. Furthermore, it proposes a *multi-position* simultaneous mutation mechanism, which significantly enhances WAF evasion efficiency and mitigates the limitations of reinforcement learning, including its reliance on large-scale data and overdependence on random selection in priority queues or MCTS.

Finally, the seven WAF pre-processing defense strategies designed by *BWAFSQLi* (such as *Normalize Letter Case* and *Comment Replacement*) further enhance system security. These strategies preprocess malicious components before they reach the WAF, significantly improving the recognizability of malicious payloads and mitigating bypass risks at the source. This not only increases WAF detection efficiency but also strengthens its layered defense capabilities against sophisticated attacks such as SQL injection, providing more reliable protection for real-world deployments.

In summary, *BWAFSQLi* achieves high bypass capability, strong compatibility, and scalability through three core advantages: innovative attack payloads, breakthroughs in semantic assurance techniques, and optimized mutation strategy search. Additionally, it provides researchers and security professionals with high-quality datasets and testing tools, substantially advancing the research and practical defense of WAFs against SQL injection attacks.

6.4 BWAFSQLi Limitations

Although *BWAFSQLi* has achieved significant results, the following limitations should be objectively noted:

(1) Adaptive limitations in SQL injection payload feature extraction:
Although integrating the *Sqlparse* syntax parsing library has significantly improved feature extraction accuracy, its original design was primarily targeted at standard SQL syntax structures. When faced with highly variant SQL injection payloads (such as those with nested multi-layer encoding or non-standard syntax), *Sqlparse*'s parsing capabilities may be limited. Future improvements could involve expanding feature extraction rules or introducing more powerful syntax parsing engines (such as ANTLR [4]) to enhance adaptability.

(2) Dependency on prior knowledge for dataset generation using context-free grammar:
Although using context-free grammar to generate the benchmark dataset ensures semantic correctness, the diversity of the generated results is limited by the coverage of the initial syntax rules. To mitigate this limitation, we publicly provide the complete CFG rule repository, enabling researchers to conveniently expand or adjust the rules to enhance the variability and coverage of the dataset.

(3) Database compatibility limitations and method universality:
The current mutation strategy is designed based on MySQL syntax characteristics and may require targeted adjustments in other database systems (such as PostgreSQL and Oracle). However, it is worth noting that the core mutation strategy search method is universally applicable across databases. By expanding the context-free grammar according to the syntax rules of the target database, strategy migration can be effectively achieved.

7 Conclusion

In the face of increasingly complex web attack environments, the efficient detection and interception capabilities of WAF are crucial—particularly against SQL injection, a common entry point frequently exploited by attackers. To address the current limitations in evaluating WAF protection effectiveness, we propose an innovative method named *BWAFSQLi*, which integrates high-quality attack payload generation, a comprehensive mutation strategy set, and an intelligent strategy selection mechanism. Our experiments cover three deep learning–based gray-box WAFs (CNN [17], LSTM [9], and GRU [22]) and eight open-source or commercial black-box WAFs (ModSecurity, Changting LeiChi, Huawei Cloud, Tianyi Cloud, Cloudflare, AWS, F5 and Fortinet). The experimental results show that BWAFSQLi increases WAFs' FNR by up to **93.39%** in gray-box settings and up to **58.49%** in black-box settings, outperforming state-of-the-art methods, respectively, and demonstrates excellent performance, requiring only **2** HTTP requests per successful bypass. These findings not only highlight critical vulnerabilities in existing WAF deployments, but also provide actionable insights that can guide WAF vendors in optimizing their detection mechanisms.

To assist WAF vendors enhance their system defenses, we further propose seven practical preprocessing defense strategies. When applied to the same eight black-box WAFs, these strategies reduced the FNR growth caused by our attack methods by an average of **74.59%**. More specifically, analysis across different request types—including GET, POST, GET(JSON), and POST(JSON)—showed that the average reduction in FNR growth was at least **66.27%**. These results clearly demonstrate the effectiveness of our proposed defenses in mitigating the risks posed by advanced SQL injection mutations.

In the future, we plan to further enhance the framework by updating the feature extraction methods, grammar rules, and mutation strategies to support a wider range of attack types (e.g., XSS, RCE). In addition, we aim to develop an integrated toolchain for SQL injection vulnerability discovery. This effort will promote the generalization of our framework toward multi-vector adversarial testing and continuously advance the intelligent evolution of web security defense systems.

Acknowledgments

This work was supported by Basic Innovation Research Cultivation Program of Yanshan University under Grant 2024LGZD004, National Natural Science Foundation of China (no.62376240), S&T Program of Hebei (no. 236Z0304G).

References

[1] Mohammadhossein Amouei, Mohsen Rezvani, and Mansoor Fateh. 2022. RAT: Reinforcement-Learning-Driven and Adaptive Testing for Vulnerability Discovery in Web Application Firewalls. IEEE Transactions on Dependable and Secure Computing 19, 5 (Sept. 2022), 3371–3386.

[2] andialbrecht. 2025. python-sqlparse: Parse SQL Statements. Technical Report. GitHub. Retrieved 2025-01-06 from https://github.com/andialbrecht/sqlparse

[3] Maksym Andriushchenko, Francesco Croce, Nicolas Flammarion, and Matthias Hein. 2020. Square Attack: A Query-Efficient Black-Box Adversarial Attack via Random Search. arXiv:1912.00049 [cs.LG] https://arxiv.org/abs/1912.00049

[4] ANTLR. 2023. ANTLR. ANTLR. Retrieved 2023-12-31 from https://www.antlr.org/

[5] Dennis Appelt, Cu D. Nguyen, and Lionel Briand. 2015. Behind an Application Firewall, Are We Safe from SQL Injection Attacks?. In Proceedings of the 2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST). IEEE Computer Society, Los Alamitos, CA, USA, 1–10. doi:10.1109/ICST.2015.7102581

[6] Dennis Appelt, Cu D. Nguyen, Annibale Panichella, and Lionel C. Briand. 2018. A Machine-Learning-Driven Evolutionary Approach for Testing Web Application Firewalls. IEEE Transactions on Reliability 67, 3 (Sept. 2018), 733–757. doi:10.1109/TR.2018.2805763

[7] Simon Applebaum, Tarek Gaber, and Ali Ahmed. 2021. Signature-Based and Machine-Learning-Based Web Application Firewalls: A Short Survey. Procedia Computer Science 189 (2021), 359–367. doi:10.1016/j.procs.2021.05.105 AI in Computational Linguistics.

[8] Rezan Bakır, I. Jemal, and O. Cheikhrouhou. 2025. UniEmbed: A Novel Approach to Detect XSS and SQL Injection Attacks Leveraging Multiple Feature Fusion with Machine Learning Techniques. Arabian Journal for Science and Engineering (2025). doi:10.1007/s13369-

024-09916-4

[9] Xianghao Chu and Zhen Liu. 2019. Research on SQL Injection Attack Detection Based on LSTM Neural Network. Journal of Tianjin University of Technology 6 (2019), 41–46. Retrieved 2024-04-20 from https://www.zhangqiaokeyan.com/academic-journal-cn_journal-tianjin-university-technology_thesis/0201273470604.html

[10] Armin Cremers and Seymour Ginsburg. 1975. Context-Free Grammar Forms. J. Comput. System Sci. 11, 1 (1975), 86–117. https://www.sciencedirect.com/science/article/pii/S0022000075800511

[11] CWE. 2024. Weaknesses in the 2024 CWE Top 25 Most Dangerous Software Weaknesses. Retrieved 2026-01-06 from https://cwe.mitre.org/top25/index.html

[12] Gonzalo De La Torre Parra, Paul Rad, Kim-Kwang Raymond Choo, and Nicole Beebe. 2020. Detecting Internet of Things Attacks Using Distributed Deep Learning. Journal of Network and Computer Applications 163 (2020), 102662. doi:10.1016/j.jnca.2020.102662

[13] Luca Demetrio, Andrea Vábbi, and Giovanni Lagorio Costa. 2020. WAF-A-MoLE: Evading Web Application Firewalls through Adversarial Machine Learning. arXiv:2001.01952 doi:10.48550/arXiv.2001.01952

[14] Ian Goodfellow, Jonathon Shlens, and Christian Szegedy. 2018. Adversarial Attacks and Defences: A Survey. arXiv. Retrieved 2018-10-01 from https://ar5iv.labs.arxiv.org/html/1810.00069

[15] Kumar J Harish and Ponsam J Godwin. 2023. Securing Web Application using Web Application Firewall (WAF) and Machine Learning. In Proceedings of the 2023 First International Conference on Advances in Electrical, Electronics and Computational Intelligence (ICAEECI) (ICAEECI '23). ACM, New York, NY, USA, 1–8. doi:10.1109/icaeeci58247.2023.10370872

[16] Mojtaba Hemmati and Mohammad Ali Hadavi. 2021. Using Deep Reinforcement Learning to Evade Web Application Firewalls. In Proceedings of the 2021 18th International ISC Conference on Information Security and Cryptology (ISCISC). 35–41. doi:10.1109/ISCISC53448.2021.9720473

[17] Ao Luo, Wei Huang, and Wenqing Fan. 2019. A CNN-Based Approach to the Detection of SQL Injection Attacks. In Proceedings of the 2019 IEEE/ACIS 18th International Conference on Computer and Information Science (ICIS). 320–324. doi:10.1109/ICIS46139.2019.8940196

[18] Chaochao Luo, Zhiyuan Tan, Geyong Min, Jie Gan, Wei Shi, and Zhihong Tian. 2021. A Novel Web Attack Detection System for Internet of Things via Ensemble Classification. IEEE Transactions on Industrial Informatics 17, 8 (Aug. 2021), 5810–5818. doi:10.1109/TII.2020.3038761

[19] MariaDB. n.d.. Random Query Generator (with MariaDB Patches). GitHub. Retrieved 2025-10-26 from https://github.com/MariaDB/randgen

[20] Morzeux. 2020. HttpParamsDataset: A Large-Scale Dataset for HTTP Parameter Inference. Morzeux. Retrieved 2023-12-30 from https://github.com/Morzeux/HttpParamsDataset

[21] K. Nagendran, S. Balaji, B. Akshay Raj, P. Chanthrika, and R. G. Amirthaa. 2020. Web Application Firewall Evasion Techniques. In Proceedings of the 2020 6th International Conference on Advanced Computing and Communication Systems (ICACCS). 194–199.

[22] niccolox. 2019. WAF-Brain - the clever and efficient Firewall for the Web. niccolox. <https://github.com/niccolox/waf-brain>

[23] OWASP. 2021. Top 10 Web Application Security Risks. Retrieved 2026-01-06 from https://owasp.org/www-project-top-ten/

[24] Stefan Prandl, Mihai Lazarescu, and Duc Son Pham. 2015. A Study of Web Application Firewall Solutions. In Proceedings of the International Conference on Information Systems Security (ICISS '15). Springer, Cham, Switzerland, 501–510. doi:10.1007/978-3-319-26961-0_29

[25] Zhenqing Qu, Xiang Ling, Ting Wang, Xiang Chen, Shouling Ji, and Chunming Wu. 2024. AdvSQLi: Generating Adversarial SQL Injections Against Real-World WAF-as-a-Service. IEEE Transactions on Information Forensics and Security 19 (2024), 2623–2638. doi:10.1109/TIFS.2024.3350911

[26] Markus Ring, Sarah Wunderlich, Deniz Scheuring, Dieter Landes, and Andreas Hotho. 2019. A Survey of Network-Based Intrusion Detection Data Sets. Computers & Security 86 (Sept. 2019), 147–167. doi:10.1016/j.cose.2019.06.005

[27] Ivan Ristić and Christian Folini. 2023. ModSecurity Handbook. Technical Report. Feisty Duck. Retrieved 2023-10-01 from https://www.feistyduck.com/books/modsecurity-handbook/

[28] Jesús-Ángel Román-Gallego, María-Luisa Pérez-Delgado, Marcos Luengo Viñuela, and María-Concepción Vega-Hernández. 2025. Artificial Intelligence Web Application Firewall for Advanced Detection of Web Injection Attacks. Expert Systems 42, 1 (2025), e13505. doi:10.1111/exsy.13505

[29] Amirmohammad Sadeghian, Mazdak Zamani, and Suhaimi Ibrahim. 2013. SQL Injection Is Still Alive: A Study on SQL Injection Signature Evasion Techniques. In Proceedings of the 2013 International Conference on Information and Communication Technology Management (ICICM '13). IEEE Computer Society, USA, 265–268. doi:10.1109/ICICM.2013.52

[30] SaneBow. 2023. WAF Evasion Environment for OpenAI Gym [Experimental]. SaneBow. Retrieved 2023-12-31 from https://github.com/SaneBow/gym-waf

[31] S. S. H. Shah. 2021. HttpParamsDataset: A Large-Scale Dataset for HTTP Parameter Inference. S. S. H. Shah. Retrieved 2023-12-30 from https://www.sikgle.com/syedsaqlainhussain/sql-injection-dataset

[32] Spanish National Research Council (CSIC) and Nature Index. 2024. Spanish National Research Council (CSIC): Institution Outputs. Nature Index. Retrieved 2024-03-15 from https://www.nature.com/nature-index/institution-outputs/spain/spanish-national-research-

council-csic/5139073734d6b65e6a002254
[33] sqlmapproject. 2023. sqlmap: Automatic SQL Injection and Database Takeover Tool. Technical Report. GitHub. Retrieved 2023-10-01 from https://github.com/sqlmapproject/sqlmap
[34] Omer Tripp, Omri Weisman, and Lotem Guy. 2013. Finding Your Way in the Testing Jungle: A Learning Approach to Web Security Testing. In Proceedings of the International Symposium on Software Testing and Analysis (ISSTA '13) (Lugano, Switzerland, July 15–20, 2013), Mauro Pezzè and Mark Harman (Eds.). ACM, 347–357. doi:10.1145/2483760.2483776
[35] Min Wan and Kun Liu. 2012. An Improved Eliminating SQL Injection Attacks Based on Regular Expressions Matching. In Proceedings of the 2012 International Conference on Control Engineering and Communication Technology. 210–212. doi:10.1109/ICCECT.2012.235
[36] Chenxu Wang, Ming Zhang, Jinjing Zhao, and Xiaohui Kuang. 2022. Black-Box Adversarial Attacks on Deep Neural Networks: A Survey. In Proceedings of the 2022 4th International Conference on Data Intelligence and Security (ICDIS). 88–93. doi:10.1109/ICDIS55630.2022.00021
[37] Hanrui Wang, Shuo Wang, Zhe Jin, Yandan Wang, Cunjian Chen, and Massimo Tistarelli. 2021. Similarity-Based Gray-Box Adversarial Attack Against Deep Face Recognition. In Proceedings of the 2021 16th IEEE International Conference on Automatic Face and Gesture Recognition (FG 2021). 1–8. doi:10.1109/FG52635.2021.9667076
[38] Xianbo Wang and Han Hu. 2020. Evading Web Application Firewalls with Reinforcement Learning. Retrieved 2020-12-14 from https://openreview.net/forum?id=m5AntlhJ7Z5 OpenReview preprint, last modified May 5, 2023.
[39] Yuqi Yu, Guannan Liu, Hanbing Yan, Hong Li, and Hongchao Guan. 2018. Attention-Based Bi-LSTM Model for Anomalous HTTP Traffic Detection. In Proceedings of the 2018 15th International Conference on Service Systems and Service Management (ICSSSM). 1–6. doi:10.1109/ICSSSM.2018.8465034
[40] Bing Zhang, Rong Ren, Jia Liu, Mingcai Jiang, Jiadong Ren, and Jingyue Li. 2024. SQLPsdem: A Proxy-Based Mechanism Towards Detecting, Locating and Preventing Second-Order SQL Injections. IEEE Transactions on Software Engineering 50, 7 (July 2024), 1807–1826. doi:10.1109/TSE.2024.3400404
[41] Long Zhang, Donghong Zhang, Chenghong Wang, Jing Zhao, and Zhenyu Zhang. 2019. ART4SQLi: The ART of SQL Injection Vulnerability Discovery. IEEE Transactions on Reliability 68, 4 (Dec. 2019), 1470–1489. doi:10.1109/TR.2019.2910285

A Appendices

A.1 Context-free Grammar Rule

A.1.1 Context-free grammar rule definition.

Table A1 describes the definition of the context-free grammar. Specifically, it explains the meanings and uses of the various symbols appearing in the production rules, including non-terminals, terminals, the concatenation operator, and the alternation operator. Using these rules, the syntactic structure of SQL-injection payloads can be generated systematically, thereby ensuring the diversity and controllability of the payloads.

Table A1. Context-free grammar rule definition.

Symbol	Descirption
::=	Represents the production symbol, followed by a set of production rules used to define how non-terminal symbols are expanded. The production rules include both non-terminal and terminal symbols.
<String>	Represents a non-terminals, appearing on both sides of the ::= symbol. Non-terminal symbols can be expanded through production rules, ultimately generating a sequence of terminal symbols.
String	Represents a terminals, appearing on the right side of the ::= symbol, and is an indivisible basic unit.
,	Appears on the right side of the ::= symbol, representing the concatenation operator used to join non-terminal and terminal symbols.
|	Appears on the right side of the ::= symbol, representing the alternation operator, which connects multiple production rules, indicating that a non-terminals can have multiple production rule options.

A.1.2 Payload Generation Rule Sets.

Rules 1–28 form the SQL-injection payload generation set, Rules 29–30 the benign set, and Rules 31–58 contain only terminal symbols; Table A4 lists functions with examples. Table A2 gives the grammar-level generation rules (non-terminals and derivations), while Table A3 lists the corresponding terminals (functions, strings, or encodings) with examples. Separating structural rules from terminal symbols reduces information density and clarifies the hierarchical structure of the payload generation method.

Table A2. SQL injection and benign payloads in generation rules.

Number	Rule
	SQL Injection Attack Context
GR1	<SQLInjection_Payload>::=<NumericContext>|<QuoteContext>|<ParenthesesContext>|<PercentContext>
GR2	<NumericContext>::=F_Digit,<space>,<NonStackInjectionContext>|F_Digit,<StackInjectionContext>
GR3	<QuoteContext>::=F_Digit,<opQuote>,<space>,<NonStackInjectionContext>,<cmt> |F_Digit,<opSquote>,<space>,<NonStackInjectionContext>,<space>,<opOr>,<space>,<opSquote>,F_Digit |F_Digit,<opDquote>,<space>,<NonStackInjectionContext>,<space>,<opOr>,<space>,<opDquote>,F_Digit |F_Digit,<opQuote>,<space>,<NonStackInjectionContext>,<StackInjectionContext>,<cmt> |F_Digit,<opQuote>,<StackInjectionContext>,<cmt> |F_Digit,<opQuote>,<space>,<StackInjectionContext>,<StackInjectionContext>,<cmt>
GR4	<ParenthesesContext>::=F_Digit,<opParC>,<NonStackInjectionContext>,<cmt> |F_Digit,<opParC>,<NonStackInjectionContext>,<opOr>,<opParO>,F_Digit |F_Digit,<opSquote>,<opParC>,<NonStackInjectionContext>,<opOr>,<opParO>,<opSquote>,F_Digit |F_Digit,<opDquote>,<opParC>,<NonStackInjectionContext>,<opOr>,<opParO>,<opDquote>,F_Digit |F_Digit,<opParC>,<NonStackInjectionContext>,<StackInjectionContext>,<cmt> |F_Digit,<opParC>,<StackInjectionContext>,<cmt> |F_Digit,<opParC>,<NonStackInjectionContext>,<StackInjectionContext>,<cmt>
GR5	<PercentContext>::=F_Digit,<opPercent>,<opQuote>,<parC>,<NonStackInjectionContext>,<cmt> |F_Digit,<opPercent>,<opQuote>,<parC>,<NonStackInjectionContext>,<StackInjectionContext>,<cmt> |F_Digit,<opPercent>,<opQuote>,<parC>,<StackInjectionContext>,<cmt> |F_Digit,<opPercent>,<opQuote>,<parC>,<NonStackInjectionContext>,<StackInjectionContext>,<cmt>
GR6	<NonStackInjectionContext>::=<UnionInjectionContext>|<TautologyInjectionContext> |<ErrorInjectionContext>|<BooleanBlindInjectionContext>|<TimeDelayBlindInjectionContext>
GR7	<StackInjectionContext>::=<opSem>,<StackInjection>|<opSem>,<StackInjection>,<StackInjectionContext> |<opSem>,<space>,<StackInjection>|<opSem>,<space>,<StackInjection>,<StackInjectionContext>
GR8	<StackInjection>::=<StackDropInjection>,|<StackInsertInjection>,|<StackUpdateInjection>, |<StackDeleteInjection>,|<StackCreateInjection>,|<StackUnionInjection>
GR9	<parC>::=<opParC>|<parC>,<opParC>
GR10	<opQuote>::=<opSquote>,<opDquote>
GR11	<cmt>::=F_Hashtag,<space>|<dcmt>,<space>|<dcmt>,<opAdd>
GR12	<space>::=F_Space|<space>,F_Space
GR13	<UnionInjectionContext>::=<opOrder>,<space>,<opBy>,<space>,<F_NotDigitPositiveZero> |<opUnion>,<space>,<opSelect>,<space>,<selectCols> |<opUnion>,<space>,<unionPostfix>,<opSelect>,<space>,<selectCols> |<opUnion>,<space>,<unionPostfix>,<opSelect>,<space>,<F_Concat> |<opUnion>,<space>,<unionPostfix>,<opParO>,<opSelect>,<space>,<selectCols>,<opParC>
GR14	<unionPostfix>::=<opAll>,<space>|<opDistinct>,<space>
GR15	<selectCols>::=<cols>|<cols>,<opComma>,<selectCols>
GR16	<cols>::=F_MysqlF|F_digitZero|F_NotDigitPositiveZero
GR17	<TautologyInjectionContext>::=<opOr>,<space>,<TautologyInjection>
GR18	<TautologyInjection>::F_tautology_number,F_tautology_string,<True_Query>
GR19	<True_Query>::=F_True_Query|<True_Query>,<space>,<opAnd>,<space>,F_True_Query
GR20	<TimeDelayBlindInjectionContext>::=<opAnd>,<space>,F_Sleep|<opAnd>,<space>,F_Benchmark |<opOr>,<space>,F_Sleep|<opOr>,<space>,F_Benchmark
GR21	<ErrorInjectionContext>::=<opOr>,<space>,F_Error
GR22	<BooleanBlindInjectionContext>::=<opOr>,<space>,F_Compare|<opOr>,<space>,F_elt_compare
GR23	<StackInsertInjection>::=<opInsert>,<space>,<opInto>,<space>,F_InsertContent |<opInsert>,<space>,<opInto>,<space>,F_InsertOrUpdateContent
GR24	<StackUpdateInjection>::=<opUpdate>,<space>,F_UpdateContent |<opUpdate>,<space>,F_InsertOrUpdateContent
GR25	<StackDeleteInjection>::=<opDelete>,<space>,<opFrom>,<space>,F_DeleteContent
GR26	<StackDropInjection>::=<opDrop>,<space>,<opTable>,<space>,F_TableName
GR27	<StackcreateInjection>::=<opCreate>,<space>,<opTable>,<space>,F_TableName
GR28	<StackUnionInjection>::=<opSelect>,<space>,<selectCols>
	Norm Attack Context
GR29	<Norm_Payload>::=<RandomString>|<RandomString>,<space>,<Norm_Payload>
GR30	<RandomString>::=F_random_word|F_NotDigitPositiveZero|F_random_mysql_reserved_keywords

A.1.3 Payload Mutation Rule Sets. Table A5 presents the rule sets directly associated with each mutation strategy. Table A6 lists the basic rule sets and terminal symbols associated with the composite rules in Table A5, while Table A7 provides examples of functions used in the rule sets.

Table A3. Terminals in generation rules.

Number	Rule	Number	Rule
GR31	<dcmt>::='–'	GR45	<opDelete>::='delete'
GR32	<opOrder>::='order'	GR46	<opInto>::='into'
GR33	<opBy>::='by'	GR47	<opWhere>::='where'
GR34	<opUnion>::='union'	GR48	<opFrom>::='from'
GR35	<opSelect>::='select'	GR49	<opIf>::='if'
GR36	<opDrop>::='drop'	GR50	<opSquote>::=" ' "
GR37	<opCreate>::='create'	GR51	<opDquote>::=' " '
GR38	<opAll>::='all'	GR52	<opParC>::=')'
GR39	<opDistinct>::='distinct'	GR53	<opParO>::='('
GR40	<opAnd>::='and'	GR54	<opComma>::=','
GR41	<opTable>::='table'	GR55	<opSem>::=';'
GR42	<opOr>::='or'|'xor'	GR56	<opPercent>::='%'
GR43	<opInsert>::='insert'	GR57	<opAdd>::='+'
GR44	<opUpdate>::='update'	GR58	<opZero>::='0'

Table A4. Functions that appear frequently in generation rules (terminals) and corresponding examples.

Function	Example
F_Digit	-2,-1,0,1,2
F_Hashtag	#
F_Space	␣
F_NotDigitPositiveZero	0,1,2
F_Concat	group_concat(column_name) from information_schema.columns where table_name='sleeves' and table_schema='doubt'
F_MysqlF	user(),database(),version()
F_digitZero	0
F_tautology_number	46 = (select 46), 63 = 63, (select 43) = 43
F_tautology_string	'83' like '83', ')'=')', 'h' like 'h'
F_True_Query	(select 1),True,2<>3
F_Sleep	sleep(1),(select if(substr(database(),98,1)='P',sleep(1),1)), (select sleep(1))
F_Benchmark	(select benchmark(1000000,rand())),benchmark(1000000,rand()), elt(left(database(),73)='y',benchmark(1000000,rand()),0)
F_Compare	substr(user(),98,1)='e',left(database(),58)='Q', ascii(left(select group_concat(password) from users,18))=67
F_elt_compare	elt(substr(version(),44,1)='p',1,0), elt(ascii(left(version(),31))=115,1,0), elt(substr(user(),93,1)='i',1,0)
F_InsertContent	users SET pass=91, users SET error=74, pass='flicker', id='spoke'
F_UpdateContent	users (name, pass) VALUES ('associates', 'cheeses') where headers=6
F_InsertOrUpdateContent	users (subsystem, subsystem) VALUES ('arrest', 'entrance')
F_DeleteContent	users where name='sale'
F_TableName	users,jobs,orders
F_random_word	tar,magnet,landings
F_random_reserved_keywords	SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, ALTER, TRUNCATE, PRIMARY, KEY, UNIQUE, FOREIGN, INDEX, WHERE, AND, OR, NOT, IN, LIKE, BETWEEN, COMMIT, ROLLBACK, SAVEPOINT, ORDER, BY, GROUP, HAVING, AS, FROM, TO, JOIN, ON, NULL, TRUE, FALSE, INT, VARCHAR, TEXT, DATE, FLOAT, DOUBLE, BOOLEAN

Table A5. Mapping between mutation rule sets and mutation strategies.

Mutation strategy	Number	Rule
Keyword Case Swapping	MR1	<KeywordCaseSwapping>::=A_Swap_Case
Whitespace to Control Character	MR2	<SpaceSubstitutionOther>::=F_Whitespace_Alternatives
Comment Rewriting	MR3	<MultilineCommentRewriting>::=<Left_Inline_Comment>,<Inline_Comment>,<Right_Inline_Comment> |<Left_Inline_Comment>,<space>,<Right_Inline_Comment>
Comment Extension	MR4	<SingleCommentRewriting>::A_SingleComment_Rewrite
Integer Encoding	MR5	<IntegerEncoding>::=A_Swap_Integer_Hex
Equal Swapping	MR6	<EqualSubstitution>::=<space>,opLike,<space>
And Logical Invariant	MR7	<AndLogicalInvariant>::=F_And,<space>,<True_Query> ,<space>,F_And,<space>
Or Logical Invariant	MR8	<OrLogicalInvariant>::=F_Or,<space>,<False_Query> ,<space>,F_Or,<space>
Comment Injection in Whitespace	MR9	<SpaceSubstitutionComment>::=<Left_Inline_Comment> ,<Inline_Comment>,<Right_Inline_Comment>
And Substitution	MR10	<AndSubstitution>::=F_And
Or Substitution	MR11	<OrSubstitution>::=F_Or
Keyword Inline Comment	MR12	<KeyWordInlineComment>::=A_Inline_Comment
Where Rewriting	MR13	<WhereRewriting>::=opWhere,<space>,<False_Query>, <space>,F_Or|opWhere,<space>,<True_Query>,<space>,F_And
Tautology Substitution	MR14	<TautologyRewriting>::=F_tautology_number |F_tautology_string|F_tautology_complex|True_Query
Quotation Mark Encoding	MR15	<QuotationEncoding>::=A_QuotationMark_Swap_Hex

Table A6. Basic rule sets and terminals in Mutation rules.

Number	Rule
MR16	<Left_Inline_Comment>::=opSlash,<Left_Inline_Comment_Asterisk>
MR17	<Left_Inline_Comment_Asterisk>::=<Left_Inline_Comment_Asterisk>,opAsterisk|opAsterisk
MR18	<Right_Inline_Comment>::=<Left_Inline_Comment_Asterisk>,opSlash
MR19	<Right_Inline_Comment_Asterisk>::=<Right_Inline_Comment_Asterisk>,opAsterisk|opAsterisk
MR20	<Inline_Comment>::=F_Inline_Comment_Random_Word|F_Inline_Comment_Benign|F_Inline_Comment_Random_Sentence
MR21	<True_Query>::=F_True_Query|<True_Query>,<space>,F_And,<space>,F_True_Query
MR22	<False_Query>::=F_False_Query|<False_Query>,<space>,F_Or,<space>,F_False_Query
MR23	<opLike>::='like'
MR24	<opWhere>::='where'
MR25	<opSlash>::='\'
MR26	<opAsterisk>::='*'

Table A7. Functions (terminals) in Mutation rules.

Function	Example
A_Swap_Case	and ⟶ AnD, union ⟶ UnION, or ⟶ Or
F_Whitespace_Alternatives	␣ ⟶ "\t", "\n", "\r", "\f", "\v", "/**/"
A_SingleComment_Rewrite	−+ ⟶ −+oceans dopes, − ⟶ − panes, # ⟶ #beam
A_Swap_Integer_Hex	1 ⟶ 0x1, 100 ⟶ 0x64, 666 ⟶ 0x29a
F_And	and, &&
F_Or	or, ||
A_Inline_Comment	from ⟶ /*!from*/, select ⟶ /*!select*/, order ⟶ /*!order*/
F_tautology_number	46 = (select 46), 63 = 63, (select 43) = 43
F_tautology_string	'83' like '83', ')'=')', 'h' like 'h'
F_tautology_complex	(select find_in_set(ord(mid(user(), 1, 1)), '114')) = (select ord('r') between 114 and 115)
A_QuotationMark_Swap_Hex	"users" ⟶ 0x7573657273, 'passwd' ⟶ 0x706173737764, "name"⟶ 0x756e616d65
F_Inline_Comment_Random_Word	+3P-, SWg, wVy
F_Inline_Comment_Benign	She enjoys reading books every evening, music, travel
F_Inline_Comment_Random_Sentence	seas retrieval, processors orders, trainers savings
F_True_Query	(select 7949), True, 4180<>7438, 2072=2072
F_False_Query	(select 0), False, 1661<>1661, 3753=1738
A_Swap_Case	and ⟶ AnD, union ⟶ UnION, or ⟶ Or

Received 18 July 2025; revised 8 November 2025; accepted 5 January 2026